CM3070 Final Report

Introduction

The project idea that I have chosen is *__CM3005 Project Idea 2: Predictive Modelling of Social Media Trend Emergence__*

Social media platforms are in a constant state of evolution, with viral trends particularly **memes** have been playing a crucial role in shaping online culture. Memes spread rapidly across various social media platforms, significantly influencing public discourse, marketing strategies, and social interactions. Understanding the factors that contribute to the emergence of memes presents a valuable challenge for businesses, researchers, and content creators alike.

This project aims to develop a **data-driven predictive model** that forecasts the **emergence of meme trends across different social media platforms** using advanced machine learning techniques. By analysing key engagement metrics from diverse datasets, the model will evaluate the likelihood of a meme going viral. Social media data will be sourced from Kaggle and through web scraping APIs, processed, and utilised to train models that predict the dynamics of trend emergence across different social media platforms.

Prototype Feature

Importing Necessary Libraries

In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os

import praw #for reddit Webscraping

from textblob import TextBlob  #for processing textual data
from datetime import datetime  #for working with dates and times
import time  #for adding delays in execution
import isodate  #required to parse ISO 8601 durations

# For Machine learning and deep learning
from sklearn.model_selection import train_test_split  #for splitting data
from sklearn.preprocessing import LabelEncoder, StandardScaler  #for feature scaling
from collections import Counter
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report, precision_score, recall_score, f1_score
from sklearn.metrics import roc_curve, confusion_matrix, mean_squared_error, r2_score, auc  #for evaluating models
from sklearn.linear_model import LogisticRegression  #for classification tasks
from sklearn.ensemble import RandomForestClassifier  #another classification model
from sklearn.metrics import RocCurveDisplay

import tensorflow as tf  #deep learning framework
from tensorflow.keras.models import Sequential  #building neural network models

from tensorflow.keras.layers import LSTM, Dense, Dropout, Bidirectional, Layer

from IPython.display import display
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score, StratifiedKFold
```

Importing The Datasets

This project will use datasets from Kaggle along with data collected from Reddit API to study how memes spread across different social media platforms. Kaggle provides datasets with useful information like meme popularity, engagement levels, and sentiment trends. At the same time, Reddit API allows for real-time data collection, helping to track how users interact with memes, what discussions take place, and how content performs over time. By combining these sources, the project will examine key factors such as user engagement, content characteristics, and sentiment changes to build a model that predicts meme virality. This approach ensures a broad and detailed understanding of how digital trends evolve.

https://www.kaggle.com/datasets/swathishekarand/global-meme-popularity-dataset

To ensure the datasets are prepared for meaningful analysis, each dataset will first be added separately and examined for inconsistencies, missing values, and unnecessary columns. The Kaggle dataset will be loaded and cleaned to retain only relevant information related to meme popularity, engagement levels, and sentiment trends.

Similarly, data collected through the YouTube API will undergo preprocessing, including handling missing values, formatting timestamps, and filtering out irrelevant content. Each cleaning step will generate a separate cleaned dataset, which will be saved as a CSV file in the *__cleaned_datasets__* folder. This ensures that intermediate versions of the datasets are properly stored and can be revisited if needed.

After cleaning each dataset individually, the project will integrate them by aligning common features such as meme ID, engagement metrics, and sentiment scores. This merging process will combine all relevant datasets into a final integrated dataset, which will be saved as the main CSV file. This final dataset will provide a comprehensive view of meme virality, ensuring consistency in data structure and improving model performance.

Data Preprocessing

Dataset 1 : Global Meme Popularity Dataset

In [25]:
```
#defines the file path
file_path = "datasets/global_meme_popularity_dataset.csv"

#loads the CSV file into a Pandas DataFrame
df_globalmemepopularity = pd.read_csv(file_path)

#shows the dataframe
df_globalmemepopularity.head()
```

Out[25]:

	Meme ID	Meme Name	Platform	Region	Date	Views	Likes	Shares	Comments	Sentiment Score
0	d55192ab-91f7-45f2-8d3e-e4d5ed49464a	Multi-tiered context-sensitive analyzer	Reddit	Asia	2024-03-24	23264	3394	1081	275	Neutral
1	93c038b2-d97d-41d9-ba3e-b44509bf54a0	Team-oriented responsive info-mediaries	TikTok	South America	2024-11-14	71261	7777	1690	450	Neutral
2	2d2a2d90-8e5d-4ef6-a70d-7708dd42d03f	Multi-lateral client-server approach	Twitter	South America	2024-07-28	30225	6106	2997	222	Neutral
3	1cd3046b-b9fe-4b84-b7c3-f018d4fe1318	Organized system-worthy attitude	Reddit	Africa	2024-12-27	40121	9612	88	38	Neutral
4	e0415fae-c10b-4eee-86ff-f84db2e710b7	Multi-tiered tertiary matrix	Instagram	Australia	2024-10-26	52282	3733	400	736	Negative

In [26]:
```
#displays the summary information about the dataframe, including column names, data types, and non-null values
df_globalmemepopularity.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   Meme ID          10000 non-null  object
 1   Meme Name        10000 non-null  object
 2   Platform         10000 non-null  object
 3   Region           10000 non-null  object
 4   Date             10000 non-null  object
 5   Views            10000 non-null  int64 
 6   Likes            10000 non-null  int64 
 7   Shares           10000 non-null  int64 
 8   Comments         10000 non-null  int64 
 9   Sentiment Score  10000 non-null  object
dtypes: int64(4), object(6)
memory usage: 781.4+ KB
```

The dataset titled **global_meme_popularity_dataset.csv**, contains about various memes and their engagement across social media platforms. It comprises ten distinct features, namely:

- *__Meme ID:__* A unique code for each meme, helping to tell them apart.
- *__Meme Name:__* The title or name of the meme, which might be different across platforms.
- *__Platform:__* The social media site where the meme appeared, like YouTube or Reddit.
- *__Region:__* The area where the meme became popular, which helps in studying trends in different locations.
- *__Date:__* The time when the meme activity was recorded, useful for tracking how trends change over time.
- *__Views:__* The number of times people have watched the meme, showing its reach.
- *__Likes:__* The total number of likes the meme has received, reflecting audience interest.
- *__Shares:__* The number of times the meme has been shared, showing how widely it is spreading.
- *__Comments:__* The number of comments people have left on the meme, which can help in understanding engagement and discussions.
- *__Sentiment Score:__* A rating of whether the overall reaction to the meme is positive, negative, or neutral.

The dataset includes 10,000 records, each providing details on how memes spread and engage users across different social media platforms.

Data Cleaning

Data cleaning is performed on the global meme popularity dataset to prepare it for analysis. It follows a structured approach:

- Column Renaming: Adjusts column names for consistency by renaming "Meme ID" to "Post ID" and "Meme Name" to "Post Content."
- Handling Missing Data: Fills categorical missing values with placeholders ("Unknown" for platforms and "No Content" for post content). Numerical columns are processed separately.
- Date Formatting: Converts the 'Date' column to a datetime format, removing invalid dates.
- Numeric Data Conversion: Ensures engagement metrics (Views, Likes, Shares, Comments) are correctly formatted as numerical values.
- Duplicate Removal: Drops redundant rows to ensure unique data entries.
- Platform Name Standardisation: Strips extra spaces and formats platform names in Title Case for uniformity.
- Content Type Classification: Assigns "Image," "Video," "Shorts," or "Unknown" based on platform and post content, ensuring proper categorization.
- Column Dropping: Removes unnecessary columns (e.g., "Region"). The Region column is removed because meme popularity depends on global online engagement, not location. Likes, shares, and comments already show user interest, so Region isn't needed.
- Engagement Level Classification: Categorises posts as Low, Medium, or High based on total interactions (likes, shares, and comments).
- Dataset Saving: Stores the cleaned dataset in the 'cleaned_datasets' folder for future use.
- Final Verification: Prints a confirmation message and previews the first few rows of the cleaned dataset to ensure correctness.

This process improves data quality, making it ready for advanced analysis, trend forecasting, and meme engagement studies.

In [30]:
```python
#step 1: renames the specific columns
df_cleaned_globalmemepopularity = df_globalmemepopularity.rename(columns={
    "Meme ID": "Post ID",
    "Meme Name": "Post Content",
})

#step 2: handles the missing values by filling categorical columns with 'Unknown', numerical with median if needed
df_cleaned_globalmemepopularity.fillna({
    "Platform": "Unknown",
    "Post Content": "No Content"
}, inplace=True)

#step 3: converts the 'Date' column to datetime format
#converts 'Date' column to datetime (if not already), then keep only the date (no time)
df_cleaned_globalmemepopularity['Date'] = pd.to_datetime(df_cleaned_globalmemepopularity['Date'], errors='coerce').dt.date

#removes rows where 'Date' is NaT (conversion failed)
df_cleaned_globalmemepopularity = df_cleaned_globalmemepopularity.dropna(subset=['Date'])

#step 4: ensures the numerical columns are correctly formatted
numeric_columns = ["Views", "Likes", "Shares", "Comments"]
df_cleaned_globalmemepopularity[numeric_columns] = df_cleaned_globalmemepopularity[numeric_columns].apply(pd.to_numeric, errors='coerce')

#step 5: removes any duplicate rows
df_cleaned_globalmemepopularity.drop_duplicates(inplace=True)

#step 6: strips any extra spaces and standardises platform names to Title Case
df_cleaned_globalmemepopularity["Platform"] = df_cleaned_globalmemepopularity["Platform"].str.strip().str.title()

#step 7: function to classify content type based on platform and post content
def classify_content(row):
    platform = row['Platform'].lower()
    name = str(row['Post Content']).lower()

    if 'tiktok' in platform:
        return 'Shorts'
    elif 'instagram' in platform:
        if 'reel' in name or 'video' in name:
            return 'Video'
        else:
            return 'Image'
    elif 'reddit' in platform:
        if 'gif' in name or 'image' in name or 'meme' in name:
            return 'Image'
        elif 'video' in name:
            return 'Video'
        else:
            return 'Image'
    elif 'twitter' in platform:
        return 'Image'
    elif '9gag' in platform:
        return 'Image'  # 9GAG is mostly meme-based images
    else:
        return 'Unknown'


#step 8: applies classification to create 'Content Type' column
df_cleaned_globalmemepopularity["Content Type"] = df_cleaned_globalmemepopularity.apply(classify_content, axis=1)

#step 9: drops unnecessary columns (add the column names you want to drop)
columns_to_drop = ["Region"]
df_cleaned_globalmemepopularity.drop(columns=columns_to_drop, inplace=True, errors='ignore')

#step 10:function to create and classify engagement level based on total interactions
def classify_engagement(row):
    total_interactions = row["Likes"] + row["Shares"] + row["Comments"]
```

```
    if total_interactions > 50000:
        return "High"
    elif total_interactions > 10000:
        return "Medium"
    else:
        return "Low"

#step 11: applies the engagement level classification
df_cleaned_globalmemepopularity["Engagement Level"] = df_cleaned_globalmemepopularity.apply(classify_engagement, axis=1)

#step 9: saves the cleaned dataset to the cleaned_datasets folder for future use
df_cleaned_globalmemepopularity.to_csv("cleaned_datasets/cleaned_globalmemepopularity.csv", index=False)

#step 10: prints confirmation message to indicate completion of cleaning
print("Dataset cleaning complete. Saved as cleaned_globalmemepopularity.csv.")

#step 11: displays the first few rows of the cleaned dataset to verify the cleaning process
df_cleaned_globalmemepopularity.head()
```

```
Dataset cleaning complete. Saved as cleaned_globalmemepopularity.csv.
```

Out[30]:

	Post ID	Post Content	Platform	Date	Views	Likes	Shares	Comments	Sentiment Score	Content Type	Engagement Level
0	d55192ab-91f7-45f2-8d3e-e4d5ed49464a	Multi-tiered context-sensitive analyzer	Reddit	2024-03-24	23264	3394	1081	275	Neutral	Image	Low
1	93c038b2-d97d-41d9-ba3e-b44509bf54a0	Team-oriented responsive info-mediaries	Tiktok	2024-11-14	71261	7777	1690	450	Neutral	Shorts	Low
2	2d2a2d90-8e5d-4ef6-a70d-7708dd42d03f	Multi-lateral client-server approach	Twitter	2024-07-28	30225	6106	2997	222	Neutral	Image	Low
3	1cd3046b-b9fe-4b84-b7c3-f018d4fe1318	Organized system-worthy attitude	Reddit	2024-12-27	40121	9612	88	38	Neutral	Image	Low
4	e0415fae-c10b-4eee-86ff-f84db2e710b7	Multi-tiered tertiary matrix	Instagram	2024-10-26	52282	3733	400	736	Negative	Image	Low

Dataset 2: Reddit Webscraping

The section below scrapes meme-related posts from Reddit using the **PRAW (Python Reddit API Wrapper)** library. It first sets up the Reddit API by providing the necessary credentials, including a client ID, client secret, and user agent. Next, it defines a list of popular subreddits that focus on memes, such as "memes," "dankmemes," and "wholesomememes."

The code includes several helper functions. The **estimate_views()** function calculates the estimated views of a post based on its score and the number of comments. The estimate_shares function estimates how many times a post has been shared, either by checking the number of crossposts or by calculating a value based on comments. The get_engagement_level function determines the overall engagement level of a post (High, Medium, or Low) based on the total interactions, which include likes, comments, views, and shares.

Next, the code loops through each subreddit and scrapes the top posts from the past year, specifically focusing on posts from 2024. For each post, it converts the creation time to a readable date format and analyses the **sentiment** of the post title using the ***TextBlob*** library. It categorises the sentiment as Positive, Negative, or Neutral based on the sentiment score. The code then estimates the views and shares, calculates the engagement level, and stores all this information in a list.

Finally, the code creates a pandas DataFrame from the collected data and saves it as a CSV file named "***reddit_meme_data_2024.csv***." It also prints a confirmation message and displays a preview of the first few rows of the DataFrame.

In [33]:

```
#step 1: sets up the reddit api using praw
reddit = praw.Reddit(
    client_id="Ez7d3vZYKx3etBdNUWpc-A",  #my reddit app client id
    client_secret="DC_3g0CCtm70tUofYmW3xxZQMt5U8w",  #my reddit app client secret
    user_agent="rebeccaFYP"  # a user agent for the requests
)

#step 2: creates a list of meme-related subreddits to scrape
subreddits = [
    "memes", "dankmemes", "AdviceAnimals", "MemeEconomy", "PrequelMemes",
    "terriblefacebookmemes", "funny", "HistoryMemes", "ComedyCemetery",
    "2meirl4meirl", "wholesomememes", "bonehurtingjuice", "me_irl", "ProgrammerHumor"
]

#step 3: defines helper functions
def estimate_views(score, comments):
    #estimates views based on score and comments
    return score + comments * 2

def estimate_shares(post):
    #estimates shares based on crossposts or comments
    return post.num_crossposts if hasattr(post, 'num_crossposts') else int(post.num_comments * 0.3)

def get_engagement_level(score, comments, views, shares):
    #determines engagement level based on total interactions
    total = score + comments + views + shares
    if total > 50000:
        return "High"
    elif total > 30000:
        return "Medium"
    else:
        return "Low"

#step 4: scrapes reddit posts
reddit_memes = []  #creates a list to store meme data

for sub in subreddits:
    try:
        count = 0  #tracks collected rows
        for post in reddit.subreddit(sub).top(time_filter='year', limit=500):
            # convert post creation time to datetime
            post_date = pd.to_datetime(post.created_utc, unit='s')
            if post_date.year == 2024:  #keeps only posts from 2024
                #analyses sentiment of the post title
                sentiment_score = TextBlob(post.title).sentiment.polarity
                sentiment_label = "Positive" if sentiment_score > 0.1 else "Negative" if sentiment_score < -0.1 else "Neutral"

                #estimates views and shares
                views = estimate_views(post.score, post.num_comments)
                shares = estimate_shares(post)
                engagement = get_engagement_level(post.score, post.num_comments, views, shares)

                #appends post data to the list
                reddit_memes.append({
                    'Post ID': post.id,
                    'Post Content': post.title,
                    'Platform': 'Reddit',
                    'Date': post_date.strftime('%Y-%m-%d'),  #ensures only YYYY-MM-DD format
                    'Views': views,
                    'Likes': post.score,
                    'Shares': shares,
                    'Comments': post.num_comments,
                    'Sentiment Score': sentiment_label,
                    'Content Type': 'Image',
                    'Engagement Level': engagement,
                })

                count += 1  #increments count
                if count >= 5000:
                    break  #stops when 5000 rows are collected

    except Exception as e:
        print(f"Error fetching posts from r/{sub}: {e}")  #prints error message if any
```

```
#step 5:creates dataframe and saves it
df_reddit = pd.DataFrame(reddit_memes)  #creates a pandas dataframe from the list
df_reddit.to_csv("datasets/reddit_meme_data_2024.csv", index=False)  #saves dataframe to csv
print("Filtered dataset saved as reddit_meme_data_2024.csv")  #prints a confirmation message

#step 6: show a preview of the data
df_reddit.head()  #displays the first few rows of the dataframe
```

```
Filtered dataset saved as reddit_meme_data_2024.csv
```

Out[33]:

	Post ID	Post Content	Platform	Date	Views	Likes	Shares	Comments	Sentiment Score	Content Type	Engagement Level
0	1egzi81	Bro got both eyes open 😭	Reddit	2024-07-31	202757	196579	31	3089	Neutral	Image	High
1	1foftqk	"Wait, that wasn't the same movie?"	Reddit	2024-09-24	132439	130925	6	757	Neutral	Image	High
2	1gycy3e	What’s that thing on its head?	Reddit	2024-11-23	120088	116518	13	1785	Neutral	Image	High
3	1grymc8	Same outcome, different motivation	Reddit	2024-11-15	118326	116250	0	1038	Neutral	Image	High
4	1ev1gkk	Don't worry, you've dune a great job	Reddit	2024-08-18	109700	108750	10	475	Positive	Image	High

In [34]:
```
df_reddit.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6628 entries, 0 to 6627
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   Post ID           6628 non-null   object
 1   Post Content      6628 non-null   object
 2   Platform          6628 non-null   object
 3   Date              6628 non-null   object
 4   Views             6628 non-null   int64 
 5   Likes             6628 non-null   int64 
 6   Shares            6628 non-null   int64 
 7   Comments          6628 non-null   int64 
 8   Sentiment Score   6628 non-null   object
 9   Content Type      6628 non-null   object
 10  Engagement Level  6628 non-null   object
dtypes: int64(4), object(7)
memory usage: 569.7+ KB
```

The Reddit web scraping dataset provides a comprehensive collection of data extracted from various subreddits, including posts, comments, and user interactions. It comprises ten distinct features, namely:

- *__Post ID:__* A unique code for each meme, helping to tell them apart.
- *__Post Name:__* The title or name of the meme, which might be different across platforms.
- *__Platform:__* The social media site where the meme appeared, in this case is Reddit.
- *__Date:__* The time when the meme activity was recorded, useful for tracking how trends change over time.
- *__Views:__* The number of times people have watched the meme, showing its reach.
- *__Likes:__* The total number of likes the meme has received, reflecting audience interest.
- *__Shares:__* The number of times the meme has been shared, showing how widely it is spreading.
- *__Comments:__* The number of comments people have left on the meme, which can help in understanding engagement and discussions.
- *__Sentiment Score:__* A rating of whether the overall reaction to the meme is positive, negative, or neutral.
- *__Content Type:__* The type of content represented by the meme, such as image, video, or text, providing context for the format of the meme.
- *__Engagement Level:__* A classification of the overall engagement with the meme, categorised as High, Medium, or Low, based on interactions like views, likes, shares, and comments.

The dataset includes 6628 records, each providing details on how memes spread and engage users across Reddit.

Data Cleaning

Data cleaning is performed on the Reddit meme dataset to ensure accuracy and consistency before analysis. It follows a structured approach:

- Duplicate Removal: Eliminates repeated entries by checking for unique Post IDs.
- Handling Missing Values: Drops rows with missing essential data such as Post Content, Platform, Views, Likes, Shares, Comments, and Sentiment Score to maintain dataset integrity.
- Date Formatting: Converts the 'Date' column to datetime format, filtering out invalid entries for better sorting and time-based analysis.
- Data Type Conversion: Ensures numeric columns (Views, Likes, Shares, Comments) are properly formatted and removes entries where conversions failed.
- Sentiment Validation: Restricts Sentiment Score values to Positive, Neutral, or Negative, ensuring consistency.
- Index Reset: Refreshes row numbering after modifications to keep the dataset structured.
- Saving Cleaned Data: Creates a directory ('cleaned_datasets') if needed, then saves the cleaned dataset as a CSV file for future use.

Finally, the script prints a summary of the cleaning process, including the number of remaining rows and a preview of the dataset, verifying the improvements before further analysis.

In [37]:
```
#step 7: data cleaning for the reddit meme dataset

#removes any duplicate rows based on 'Post ID' to ensure unique posts
df_reddit.drop_duplicates(subset=['Post ID'], inplace=True)

#checks for missing values in critical columns and drop rows with missing values
df_reddit.dropna(subset=['Post ID', 'Post Content', 'Platform', 'Date', 'Views', 'Likes', 'Shares', 'Comments', 'Sentiment Score'], inplace=True)

#converts the date column to datetime type for consistency and easy filtering
df_reddit['Date'] = pd.to_datetime(df_reddit['Date'], format='%Y-%m-%d', errors='coerce')

#removes any rows where date conversion failed (invalid dates)
df_reddit = df_reddit[df_reddit['Date'].notnull()]

#converts numerical columns to appropriate integer types
numeric_cols = ['Views', 'Likes', 'Shares', 'Comments']
df_reddit[numeric_cols] = df_reddit[numeric_cols].apply(pd.to_numeric, errors='coerce')

#drop rows where numeric conversion failed (NaN values in these columns)
df_reddit.dropna(subset=numeric_cols, inplace=True)

#resets index after cleaning
df_reddit.reset_index(drop=True, inplace=True)

#ensures 'Sentiment Score' column contains only expected categories (Positive, Neutral, Negative)
valid_sentiments = ['Positive', 'Neutral', 'Negative']
df_reddit = df_reddit[df_reddit['Sentiment Score'].isin(valid_sentiments)]

#creates the directory if it doesn't exist
output_directory = 'cleaned_datasets'
os.makedirs(output_directory, exist_ok=True)

#saves the cleaned DataFrame as a CSV file
cleaned_file_path = os.path.join(output_directory, 'cleaned_reddit_meme_data.csv')
df_reddit.to_csv(cleaned_file_path, index=False)  #saves without the index
print(f"Cleaned dataset saved as {cleaned_file_path}")

#prints summary of cleaned data
print(f"Data cleaned: {len(df_reddit)} rows remaining after cleaning.")

#preview cleaned dataframe
df_reddit.head()
```

```
Cleaned dataset saved as cleaned_datasets/cleaned_reddit_meme_data.csv
Data cleaned: 6628 rows remaining after cleaning.
```

Out[37]:

	Post ID	Post Content	Platform	Date	Views	Likes	Shares	Comments	Sentiment Score	Content Type	Engagement Level
0	1egzi81	Bro got both eyes open 😭	Reddit	2024-07-31	202757	196579	31	3089	Neutral	Image	High
1	1foftqk	"Wait, that wasn't the same movie?"	Reddit	2024-09-24	132439	130925	6	757	Neutral	Image	High
2	1gycy3e	What's that thing on its head?	Reddit	2024-11-23	120088	116518	13	1785	Neutral	Image	High
3	1grymc8	Same outcome, different motivation	Reddit	2024-11-15	118326	116250	0	1038	Neutral	Image	High
4	1ev1gkk	Don't worry, you've dune a great job	Reddit	2024-08-18	109700	108750	10	475	Positive	Image	High

Dataset Integration

A series of operations is peformed to combine and clean two datasets: **cleaned_globalmemepopularity.csv** and **cleaned_reddit_meme_data.csv**. By retrieving the cleaned datasets from the *cleaned_datasets folder*, it is essential for maintaining consistency in the data processing pipeline, as it reduces the risk of introducing errors from raw data. This also enhances efficiency by saving time and computational resources, as there is no need to repeat the earlier cleaning steps for datasets that have already been processed everytime the file is reopened and run.

Initially, it defines the file paths for these cleaned datasets and loads them into separate Pandas DataFrames.

The code then vertically combines these DataFrames into a single DataFrame called combined_df using the ***pd.concat()*** function, ensuring that the index is reset and columns are not sorted. Following this, the 'Date' column is converted to a datetime format, with any parsing errors coerced into NaT (Not a Time).

The code extracts the 'Year', 'Month', and 'Day' components from the 'Date' column and creates new columns for each. It reformats the 'Date' column to exclude the time component, ensuring a standard date format. The cleaned and combined dataset is then saved to a new CSV file named combined_dataset.csv in the cleaned_datasets folder, with the row indices excluded. Finally, the first few rows of the combined DataFrame are displayed.

In [2]:
```
#defines the file path
file_path = "cleaned_datasets/cleaned_globalmemepopularity.csv"
#loads the CSV file into a Pandas DataFrame
df_cleaned_globalmemepopularity = pd.read_csv(file_path)

#defines the file path
file_path = "cleaned_datasets/cleaned_reddit_meme_data.csv"
#loads the CSV file into a Pandas DataFrame
df_cleaned_reddit = pd.read_csv(file_path)

#step 1: combines the two datasets vertically and save
combined_df = pd.concat(
    [df_cleaned_globalmemepopularity, df_cleaned_reddit],
    ignore_index=True,
    sort=False
)

#step 2: converts 'Date' column to datetime format (safe parsing)
combined_df['Date'] = pd.to_datetime(combined_df['Date'], errors='coerce')

#step 3: extracts Year, Month, Day into separate columns
combined_df['Year'] = combined_df['Date'].dt.year
combined_df['Month'] = combined_df['Date'].dt.month
combined_df['Day'] = combined_df['Date'].dt.day

#step 4: sets the proper format for the 'Date' column to exclude the time component
combined_df['Date'] = combined_df['Date'].dt.strftime('%Y-%m-%d')

#saves the cleaned dataset
combined_df.to_csv("cleaned_datasets/combined_dataset.csv", index=False)
print("Cleaned dataset saved as combined_dataset.csv")

#displays the first few rows
combined_df.head()
```

```
Cleaned dataset saved as combined_dataset.csv
```

Out[2]:

	Post ID	Post Content	Platform	Date	Views	Likes	Shares	Comments	Sentiment Score	Content Type	Engagement Level	Year	Month	Day
0	d55192ab-91f7-45f2-8d3e-e4d5ed49464a	Multi-tiered context-sensitive analyzer	Reddit	2024-03-24	23264	3394	1081	275	Neutral	Image	Low	2024	3	24
1	93c038b2-d97d-41d9-ba3e-b44509bf54a0	Team-oriented responsive info-mediaries	Tiktok	2024-11-14	71261	7777	1690	450	Neutral	Shorts	Low	2024	11	14
2	2d2a2d90-8e5d-4ef6-a70d-7708dd42d03f	Multi-lateral client-server approach	Twitter	2024-07-28	30225	6106	2997	222	Neutral	Image	Low	2024	7	28
3	1cd3046b-b9fe-4b84-b7c3-f018d4fe1318	Organized system-worthy attitude	Reddit	2024-12-27	40121	9612	88	38	Neutral	Image	Low	2024	12	27
4	e0415fae-c10b-4eee-86ff-f84db2e710b7	Multi-tiered tertiary matrix	Instagram	2024-10-26	52282	3733	400	736	Negative	Image	Low	2024	10	26

In [5]:
```
combined_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16628 entries, 0 to 16627
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   Post ID          16628 non-null  object
 1   Post Content     16628 non-null  object
 2   Platform         16628 non-null  object
 3   Date             16628 non-null  object
 4   Views            16628 non-null  int64 
 5   Likes            16628 non-null  int64 
 6   Shares           16628 non-null  int64 
 7   Comments         16628 non-null  int64 
 8   Sentiment Score  16628 non-null  object
 9   Content Type     16628 non-null  object
 10  Engagement Level  16628 non-null  object
 11  Year             16628 non-null  int32 
 12  Month            16628 non-null  int32 
 13  Day              16628 non-null  int32 
dtypes: int32(3), int64(4), object(7)
memory usage: 1.6+ MB
```

The **combined_df** contains a total of 16,628 entries, each representing a unique Meme Post. It includes 14 columns that provide detailed information about each post. The key columns are:

- ***Post ID:*** A unique identifier for each post.
- ***Post Name:*** The title of the post, which gives insight into its content.
- ***Platform:*** Indicates that the posts are sourced from Reddit.
- ***Date:*** The date when the post was made, which is crucial for tracking trends over time.
- ***Views, Likes, Shares, and Comments:*** These numerical columns reflect the level of engagement each post received, showing how popular and interactive the content is.
- ***Sentiment Score:*** This column categorizes the overall reaction to the post as positive, negative, or neutral.
- ***Content Type:*** Identifies the format of the post, such as image, video, or text.
- ***Engagement Level:*** A classification that indicates how well the post is performing in terms of user interaction.
- ***Year, Month, and Day:*** These columns facilitate time-based analysis.

All columns are free of missing values, ensuring the dataset is reliable for further analysis. Overall, this cleaned dataset is a valuable resource for understanding meme trends and user engagement on the dataset.

Identifying key numeric and categorical columns

In [194…
```
# identifys the key numeric and categorical columns
numeric_cols = combined_df.select_dtypes(include=[np.number]).columns
categorical_cols = combined_df.select_dtypes(include=['object']).columns

# prints the lists of numeric and categorical columns
print("Numeric Columns:", numeric_cols.tolist())
print("Categorical Columns:", categorical_cols.tolist())
```

```
Numeric Columns: ['Views', 'Likes', 'Shares', 'Comments', 'Year', 'Month', 'Day']
Categorical Columns: ['Post ID', 'Post Content', 'Platform', 'Date', 'Sentiment Score', 'Content Type', 'Engagement Level']
```

Statistical Summary of Numeric Columns

In [197…
```
print("\nStatistical Summary of Numeric Columns:")
combined_df.describe()
```

```
Statistical Summary of Numeric Columns:
```

Out[197…

	Views	Likes	Shares	Comments	Year	Month	Day
count	16628.000000	16628.000000	16628.000000	16628.000000	16628.000000	16628.000000	16628.000000
mean	37361.692266	9746.328181	1529.332331	439.154138	2024.045947	7.355304	15.906483
std	29692.198795	12684.732665	1667.459036	471.850001	0.209376	3.203047	8.896570
min	0.000000	0.000000	0.000000	0.000000	2024.000000	1.000000	1.000000
25%	10931.000000	3228.000000	1.000000	121.000000	2024.000000	5.000000	8.000000
50%	29946.000000	6349.000000	874.500000	339.000000	2024.000000	8.000000	16.000000
75%	61109.500000	9606.000000	2974.000000	672.000000	2024.000000	10.000000	24.000000
max	202757.000000	196579.000000	4999.000000	15538.000000	2025.000000	12.000000	31.000000

Exploratory Data Analysis

Correlation Heatmap for numeric variables

This heatmap displays the correlations between different features in your dataset after preprocessing. The colours represent the strength of these relationships where darker shades mean stronger positive correlations, while lighter or neutral tones show weak or no relationship.

Key Observations:

- Likes and Shares show a strong positive correlation suggesting that when a meme receives more likes, it also tends to be shared more often. This reinforces the idea that user appreciation often leads to content being spread further.
- Shares and Comments also have a moderate positive link, which means engaging content not only gets shared but also sparks discussion.
- Sentiment Score has only a weak relationship with Likes and Shares. This implies that while sentiment may affect how people feel, it doesn't clearly drive actual engagement.
- Content Type and Engagement Level appear to have very low correlations with other features, indicating they don't interact much with direct engagement metrics. This supports your earlier finding that these features hold little predictive weight.

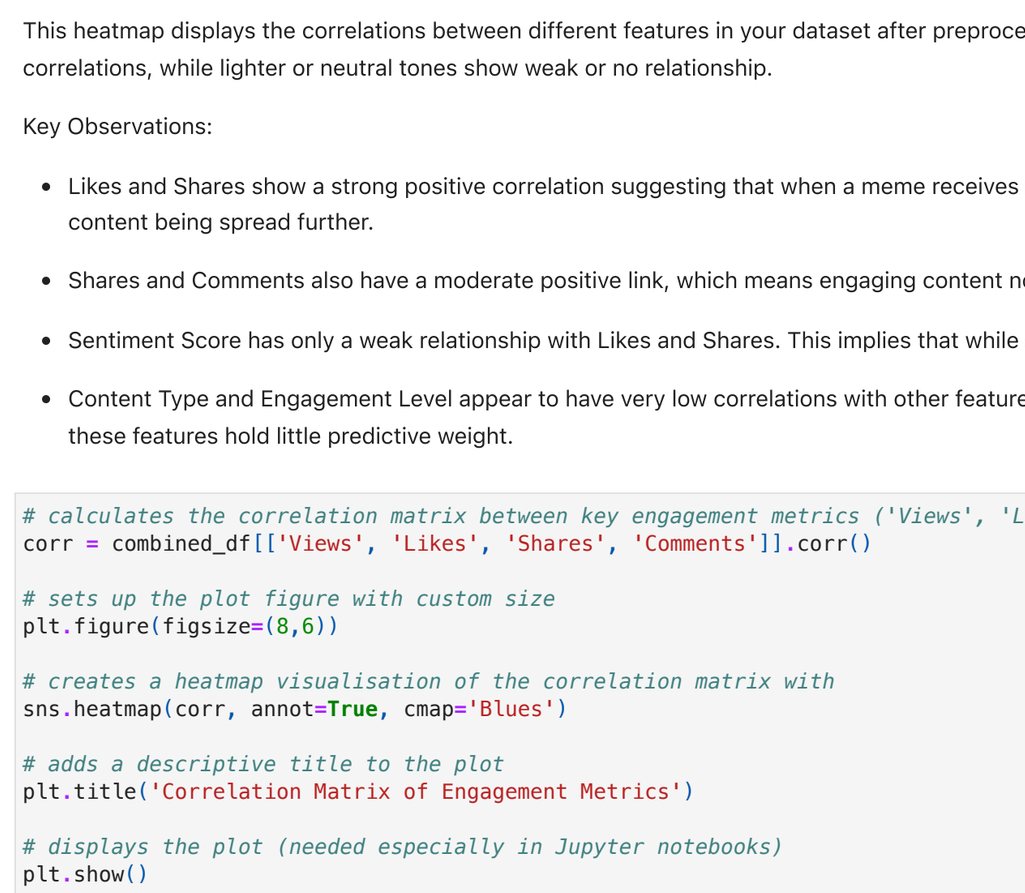
In [69]:
```
# calculates the correlation matrix between key engagement metrics ('Views', 'Likes', 'Shares', 'Comments')
corr = combined_df[['Views', 'Likes', 'Shares', 'Comments']].corr()

# sets up the plot figure with custom size
plt.figure(figsize=(8,6))

# creates a heatmap visualisation of the correlation matrix with
sns.heatmap(corr, annot=True, cmap='Blues')

# adds a descriptive title to the plot
plt.title('Correlation Matrix of Engagement Metrics')

# displays the plot (needed especially in Jupyter notebooks)
plt.show()
```



Sentiment vs Engagement Boxplot

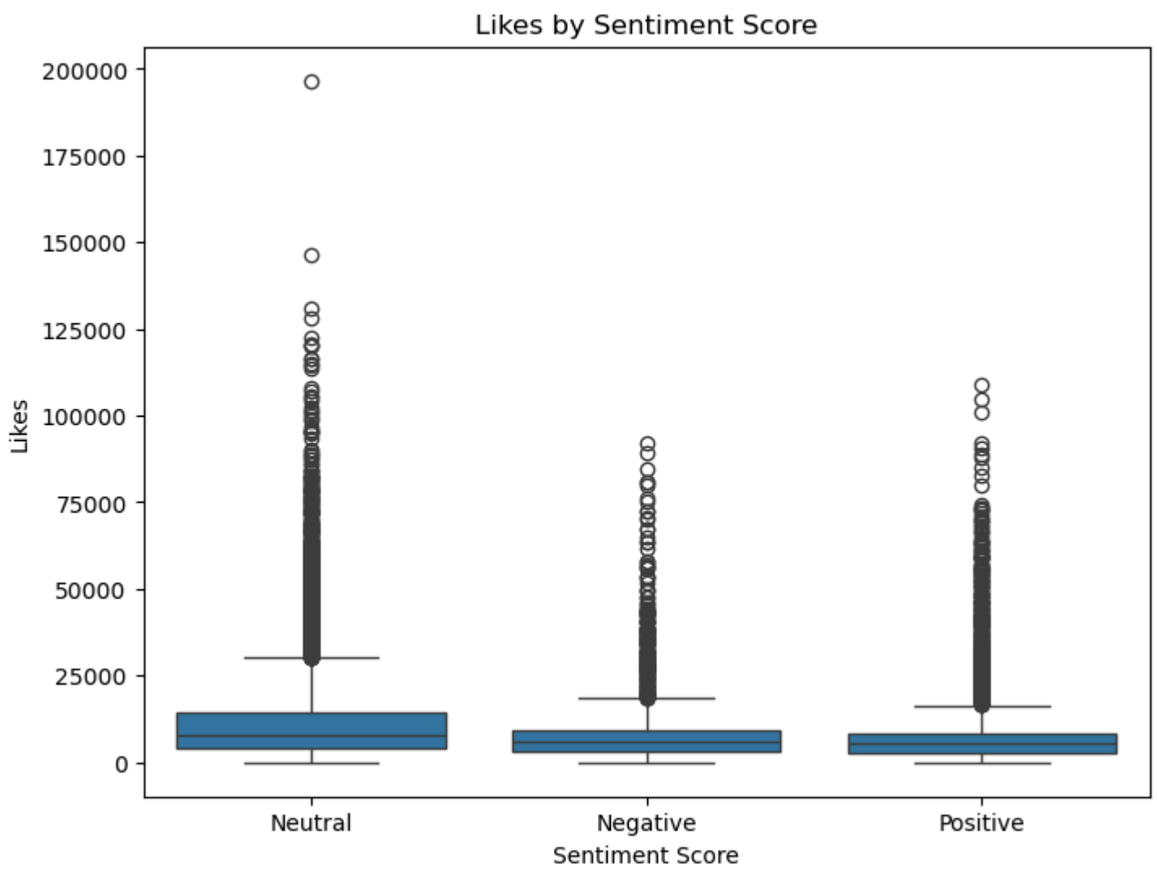
The boxplot shows how the number of likes varies depending on the sentiment of the posts, whether they're Neutral, Negative, or Positive. Surprisingly, posts with a Neutral sentiment have the highest median number of likes, meaning they tend to perform better on average than Positive or Negative ones. While one might expect Positive posts to attract more likes, this result suggests that Neutral content may be more broadly appealing or less divisive, leading to wider acceptance. All three sentiment categories show a wide spread of values, with many outliers, which means that some posts received far more likes than the rest, likely due to going viral or hitting trending topics. This finding supports the idea that sentiment alone isn't a strong predictor of engagement, but Neutral-toned content can still perform surprisingly well across audiences.

In [71]:
```
# creates the new figure with the specified size
plt.figure(figsize=(8, 6))

# creates the boxplot to visualise the distribution of 'Likes' for each 'Sentiment Score'
sns.boxplot(x='Sentiment Score', y='Likes', data=combined_df)

# sets the title of the plot
plt.title('Likes by Sentiment Score')
```

```
# displays the plot
plt.show()
```



Pairplot for Relationships Between Top Numeric Features

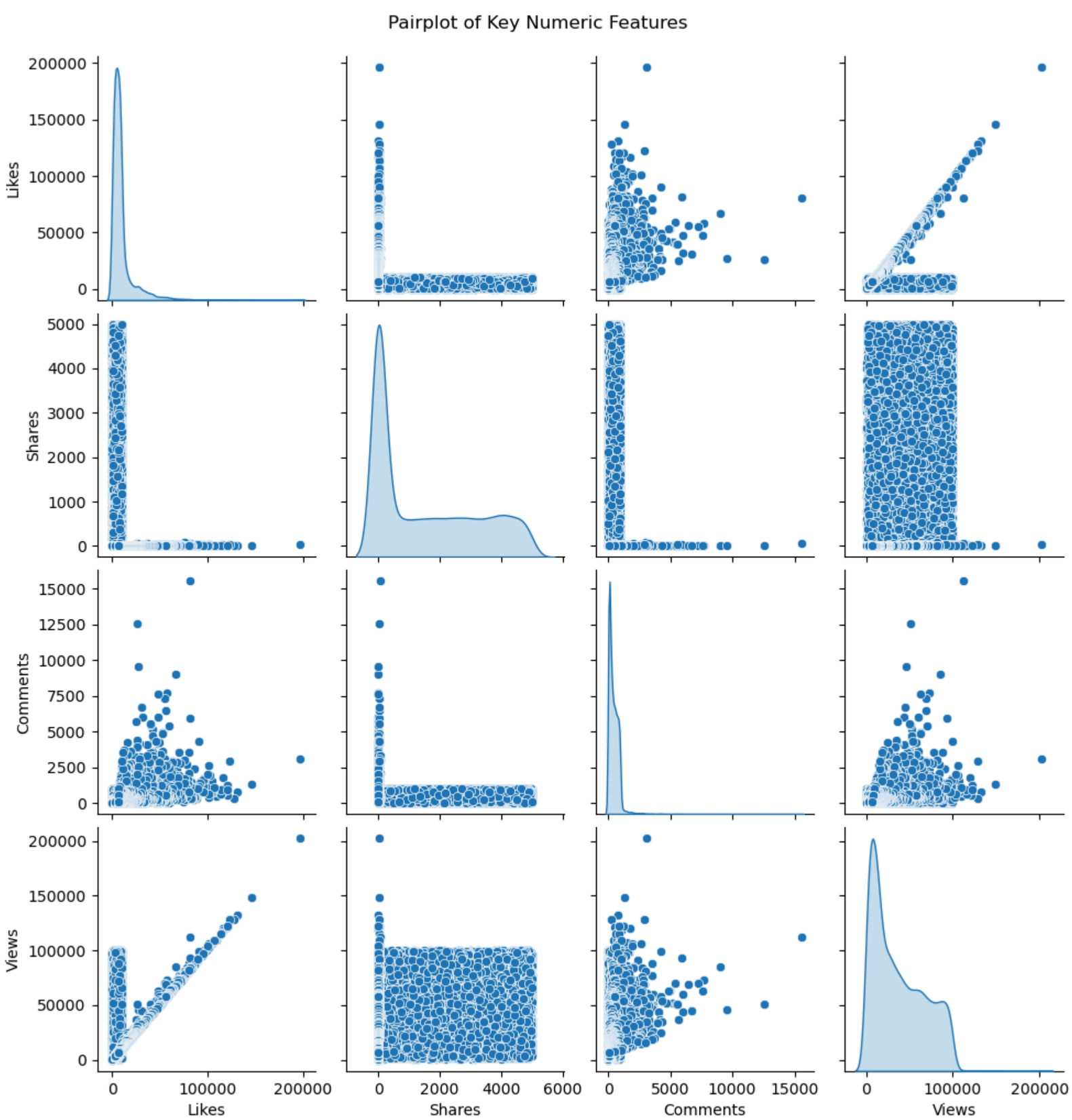
The pairplot shows how the four key features, **Likes, Shares, Comments, and Views** relate to each other and how they're spread out across the dataset. The diagonal histograms reveal that all four features are right-skewed, meaning most posts have low values, and only a few receive very high engagement, hinting at occasional viral activity. The scatter plots show a clear positive correlation between Likes and Shares, suggesting that the more people like a post, the more likely they are to share it too. Similarly, Likes and Views and Shares and Views also show some clustering in the higher ranges, implying that visibility and appreciation go hand-in-hand. However, Comments appear more scattered, showing weaker or inconsistent relationships with the other features, which may suggest that commenting behaviour is more unpredictable or driven by other factors.

In [427…

```
# generates the pairplot for selected numeric columns from combined_df
sns.pairplot(combined_df[['Likes', 'Shares', 'Comments', 'Views']], diag_kind='kde')

# adds a super title to the entire plot
plt.suptitle('Pairplot of Key Numeric Features', y=1.02)

# displays the complete plot
plt.show()
```



Time Series Analysis: Engagement Trends Over Time

The line graph titled "7-Day Rolling Average of Engagement Metrics Over Time" offers a clear view of how user engagement—measured through likes, shares, and comments—has changed from March 2024 to January 2025. Each metric is represented by a distinct coloured line, blue for likes, orange for shares, and green for comments. The use of a 7-day rolling average smooths out short-term fluctuations, making long-term trends easier to spot.

From the graph, it can be suggested that likes consistently show the highest values across the entire period, suggesting that users are more inclined to express approval than to share or comment. This could reflect the ease and immediacy of liking content compared to other forms of interaction. Shares and comments follow similar patterns but at lower levels, with occasional spikes that may correspond to viral posts or trending topics.

The steady rise and fall of these metrics over time also hint at seasonal or campaign-driven engagement. For instance, noticeable peaks in likes and shares around certain months may align with holidays, events, or marketing pushes. The rolling average helps highlight these patterns without being distracted by daily noise.

Hence, providing valuable insight into how engagement evolves, helping content creators and marketers identify optimal posting periods and understand which types of interaction dominate over time.

In [41]:
```python
# groups by 'Date' and calculate the mean of 'Likes', 'Shares', and 'Comments'
# resets the index to convert the grouped data back into a DataFrame
time_series = combined_df.groupby('Date')[['Likes', 'Shares', 'Comments']].mean().reset_index()

# covnerts the 'Date' column to datetime format for proper plotting
time_series['Date'] = pd.to_datetime(time_series['Date'])

# sets the figure size for the plot
plt.figure(figsize=(12, 6))

# loops through each engagement metric to plot its 7-day rolling average
for col in ['Likes', 'Shares', 'Comments']:
    # Calculate the rolling mean with a window of 7 days and plot it against the 'Date'
    plt.plot(time_series['Date'], time_series[col].rolling(7).mean(), label=col)

# sets the title of the plot
plt.title('7-Day Rolling Average of Engagement Metrics Over Time')

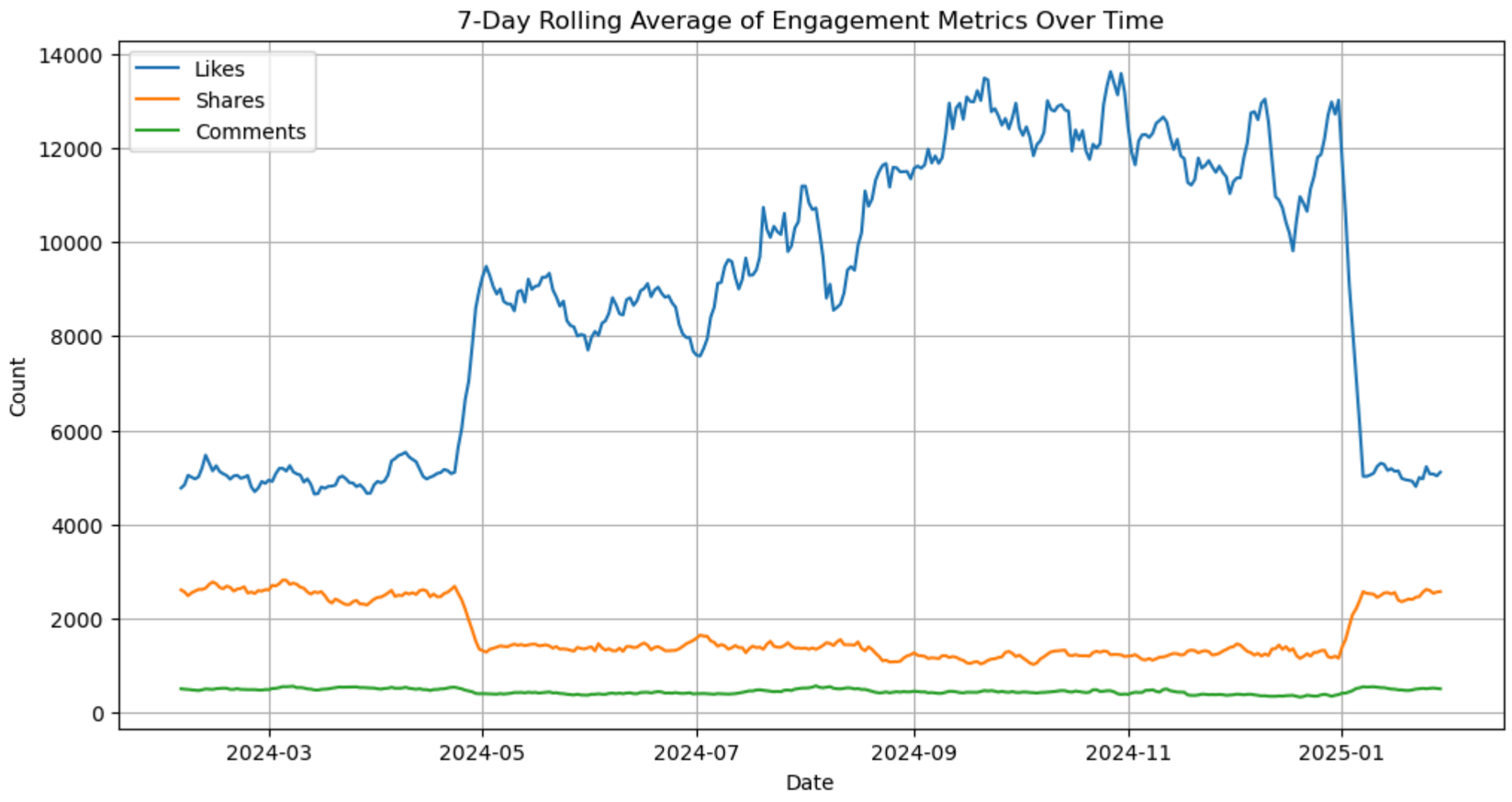
# labels the x-axis
plt.xlabel('Date')

# labels the y-axis
plt.ylabel('Count')

# displays the legend to identify each line
plt.legend()

# allows grid lines for better readability
plt.grid(True)

# displays the plot
plt.show()
```



Yearly and Monthly Distribution of Social Media Engagement Metrics

The box plots provide a clear visual comparison of how social media engagement metrics namely, **Likes, Shares, Comments, and View** are distributed across the different years and months in the dataset. Each metric is shown in two separate plots, one for yearly trends and one for monthly patterns.

From the yearly plots, it's evident that engagement was significantly higher in 2024 compared to 2025. For instance, the median number of likes in 2024 was around 25,000, with some posts reaching up to 200,000. In contrast, 2025 shows a sharp drop, with the median close to zero and far fewer outliers. This suggests a possible shift in platform activity, content strategy, or audience behaviour between the two years.

The same pattern is seen in views and comments where 2024 had higher medians and more extreme outliers, while 2025 appears quieter. Shares, however, show a slightly different trend. Although 2024 had a median around 2,000, 2025's median is slightly higher at around 3,000, indicating that users may have been more inclined to redistribute content even if overall engagement was lower.

Looking at the monthly distributions, the engagement metrics appear relatively stable throughout the year. Likes, comments, and views tend to cluster around similar median values across all twelve months, indicating that user interaction with content remains fairly consistent regardless of season. This suggests that there are no strong seasonal effects influencing these types of engagement, and users are likely interacting with content at a steady rate year-round.

However, the presence of regular outliers, especially in likes and views points to occasional spikes in activity, likely driven by viral posts or trending topics. These outliers are scattered across different months, which implies that viral content can emerge at any time, not just during specific periods like holidays or major events.

Shares, on the other hand, show slightly more variation from month to month. Some months, such as March and December, may exhibit higher median shares, possibly due to increased content circulation during festive seasons or campaign periods. Despite this, the overall pattern remains consistent, with no extreme fluctuations that would suggest a major shift in user behaviour.

In summary, the box plots reveal that the months of 2024 was a more active year for user engagement, with higher medians and more frequent spikes. Monthly trends are relatively steady, which may reflect consistent user behaviour throughout the year. It is also important to note that the data available for 2025 is smaller in volume compared to that of 2024, which naturally affects the distribution and reduces the likelihood of capturing high-engagement outliers. With fewer posts or interactions recorded, the metrics for 2025 may not fully reflect actual user behaviour across the entire year.

In [77]:
```python
# ensures that the 'Date' column is in datetime format
combined_df['Date'] = pd.to_datetime(combined_df['Date'])

# extracts year and month from 'Date' column
combined_df['Year'] = combined_df['Date'].dt.year  # Extract the year
combined_df['Month'] = combined_df['Date'].dt.month_name()  # Extract the month name

# selects the metrics to compare
metrics = ['Likes', 'Shares', 'Comments', 'Views']  # Define the metrics for analysis

# sets the figure size for the plots
plt.figure(figsize=(18, 16))  # Adjust the figure size for better visibility

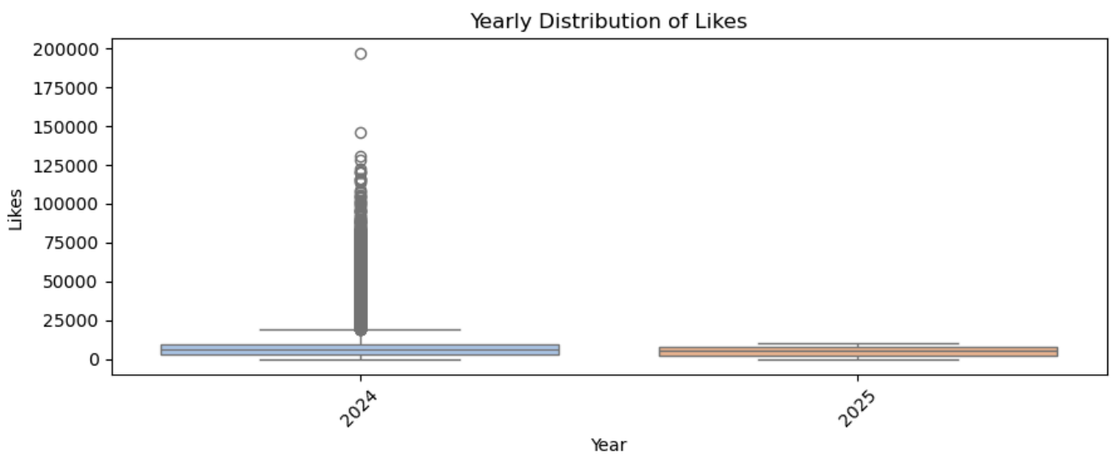
# creates subplots for yearly and monthly distribution
for i, metric in enumerate(metrics):
    # sets the yearly distribution
    plt.subplot(4, 2, i * 2 + 1)  # creates a subplot for yearly distribution
    sns.boxplot(x='Year', y=metric, hue='Year', data=combined_df, palette='pastel', legend=False)
```

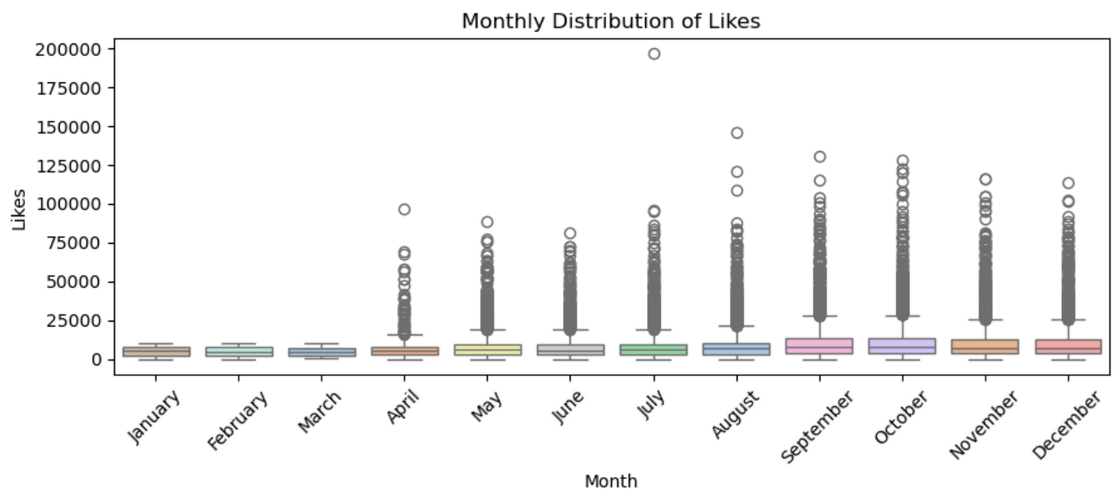
```
    plt.title(f'Yearly Distribution of {metric}')  # sets the title for the yearly plot
    plt.xticks(rotation=45)  # rotates x-axis labels for better readability

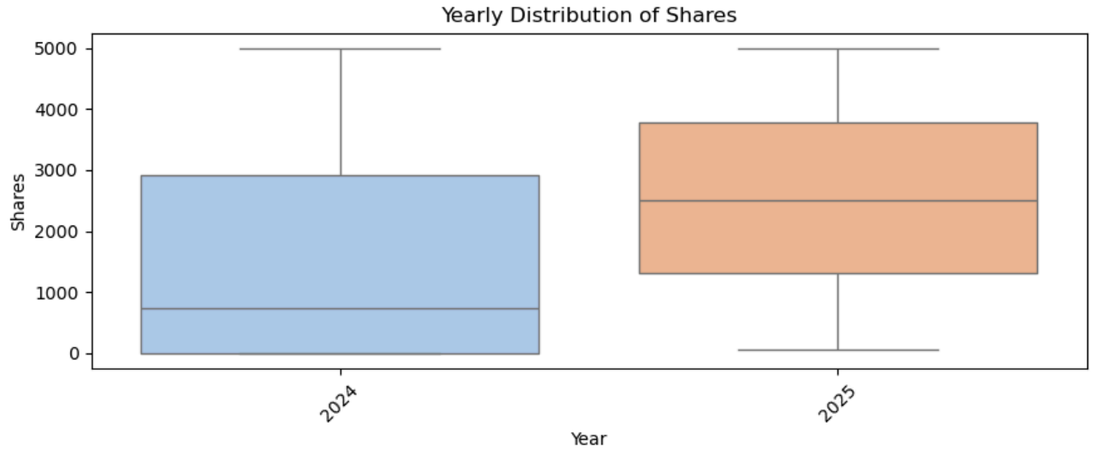
    # sets the Monthly distribution
    plt.subplot(4, 2, i * 2 + 2)  # creates a subplot for monthly distribution
    sns.boxplot(
        x='Month', y=metric, hue='Month', data=combined_df,
        palette='pastel', legend=False,
        order=[
            'January', 'February', 'March', 'April', 'May', 'June',
            'July', 'August', 'September', 'October', 'November', 'December'
        ]
    )
    plt.title(f'Monthly Distribution of {metric}')  # sets the title for the monthly plot
    plt.xticks(rotation=45)  # rotates x-axis labels for better readability

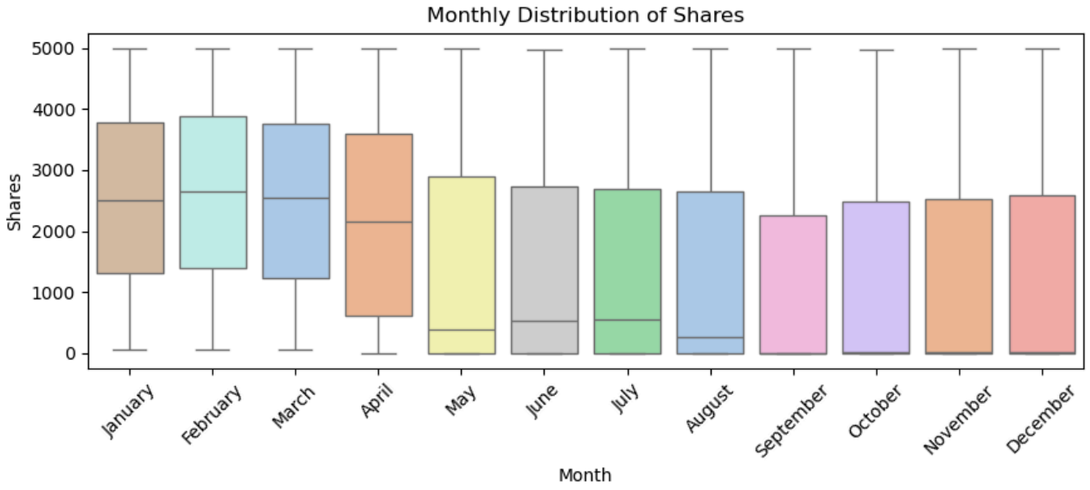
# adjusts layout to prevent overlap of subplots
plt.tight_layout()

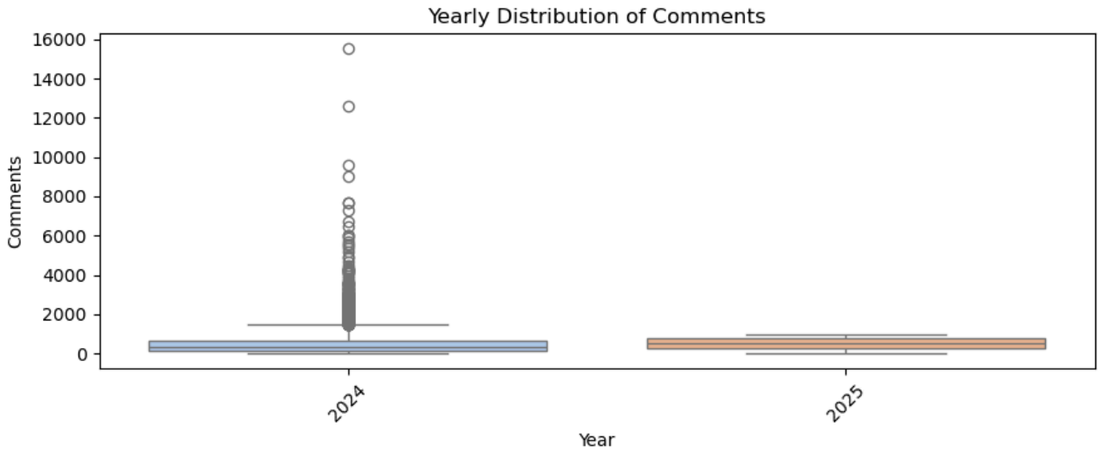
# displays the plots
plt.show()
```

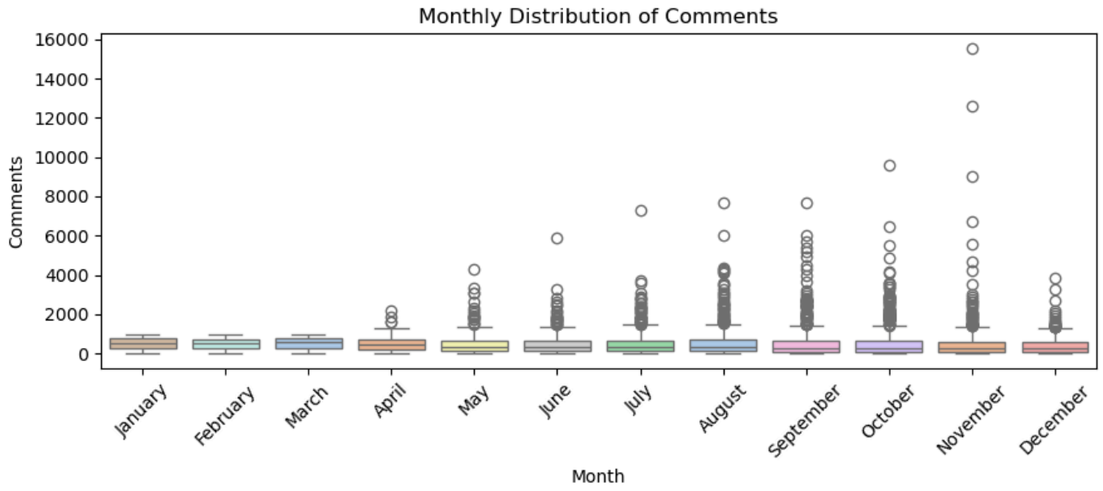


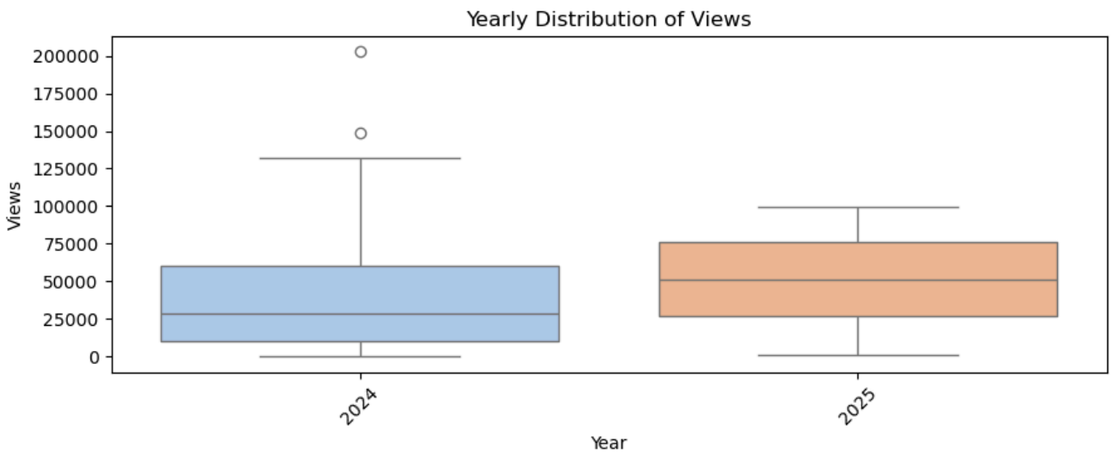


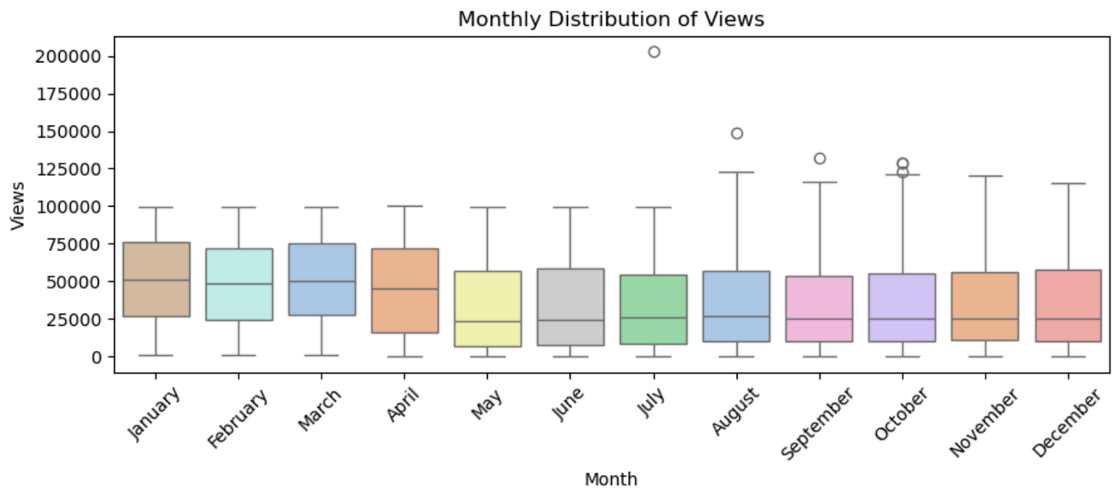












Developing And Training the Models

Classification Model

The use of classification models are designed to categorise the data into predefined groups based on input features. I have decided to use Logistic Regression and Random Forest as the classification methods to predict whether a meme will trend or not. ***Logistic Regression*** is a statistical model that estimates probabilities based on the relationship between independent variables, making it effective for binary classification tasks. It calculates weighted sums of features and applies a sigmoid function to determine the likelihood of an event occurring, such as meme virality.

Why was Logistic Regression Chosen:

- Chosen for its effectiveness in binary classification tasks, particularly for estimating probabilities.
- Utilises a statistical approach that calculates weighted sums of input features, making it interpretable and straightforward.
- Applies a sigmoid function to determine the likelihood of an event, such as whether a meme will go viral.
- Suitable for scenarios where the relationship between independent variables is linear, providing a clear understanding of feature impacts.

On the other hand, ***Random Forest*** is an ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. Each tree is trained on a subset of the data, and the final prediction is based on the majority vote of all trees. This model is particularly useful for handling complex, nonlinear relationships between features, making it well-suited for social media trend analysis.

Why was Random Forest Chosen:

- Selected for its ability to handle complex, nonlinear relationships between features, which is common in social media data.
- An ensemble learning method that constructs multiple decision trees, enhancing prediction accuracy and reducing the risk of overfitting.
- Each tree is trained on a random subset of the data, allowing for diverse perspectives in predictions and improving robustness.
- Particularly effective in capturing interactions between features, making it well-suited for analysing meme trends.

Evaluation Metrics:

To evaluate both models, **accuracy** and **AUC (Area Under the Curve)** scores will be calculated. Accuracy measures the proportion of correct predictions, while AUC evaluates the model's ability to distinguish between trending and non-trending memes. In addition to these metrics, **precision, recall, and F1 score** will also be assessed to provide a more comprehensive evaluation of the models' performance.

Precision measures the proportion of true positive predictions among all positive predictions made by the model, indicating how many of the memes predicted to trend actually did trend. High precision is crucial in scenarios where false positives are costly, ensuring that the model is reliable in its positive predictions.

Recall, or sensitivity, assesses the model's ability to identify all relevant instances, specifically the proportion of true positives among all actual positive cases. High recall means that the model successfully identifies most of the trending memes, which is important for capturing as many relevant trends as possible.

The F1 score, the harmonic mean of precision and recall, provides a single metric that balances both aspects. It is particularly useful when dealing with imbalanced datasets, where one class may be more prevalent than the other. A high F1 score indicates that the model maintains a good balance between precision and recall, making it a valuable measure for evaluating overall model performance. By comparing these metrics, it can be determined which classification model performs best in predicting meme virality.

Loads The Dataset for Model Training

In [15]:
```
#loads dataset
df_memes = pd.read_csv("cleaned_datasets/combined_dataset.csv")
```

Step 1 : Define Features & Target

A new label called 'Trend' is created to classify whether a meme is trending or not. The process begins by calculating the total engagement for each meme, which is the sum of its views, likes, shares, and comments. The median value of total engagement across the dataset is then determined. Memes with engagement above the median are assigned a **label of '1' (indicating they are trending)**, while those below or equal to the median receive a **label of '0' (indicating they are not trending)**. This method provides a more balanced class distribution, which is beneficial when training machine learning models, as it reduces the risk of the model being biased towards one class.

This provides a more balanced class distribution, which is beneficial when training machine learning models, as it reduces the risk of the model being biased towards one class. Class balance is crucial in machine learning because imbalanced datasets can lead to models that perform well on the majority class but poorly on the minority class. This can result in high accuracy but low predictive power for the less frequent class, which is often the class of interest. By ensuring a balanced distribution, the model can learn to recognize patterns in both classes effectively, leading to improved overall performance and reliability in real-world applications.

In [16]:
```
# calculates the total engagement
df_memes['Total_Engagement'] = df_memes['Views'] + df_memes['Likes'] + df_memes['Shares'] + df_memes['Comments']

# creates the Trend label based on median (50th percentile)
threshold = df_memes['Total_Engagement'].quantile(0.5)
df_memes['Trend'] = (df_memes['Total_Engagement'] > threshold).astype(int)

# prints class balance
print("Trend Label Distribution:\n", df_memes['Trend'].value_counts())

# prints the explanation
print("Explanation:")
print("0 → Meme has engagement BELOW the median (Not trending)")
print("1 → Meme has engagement ABOVE the median (Trending)")
```

```
Trend Label Distribution:
 Trend
0    8314
1    8314
Name: count, dtype: int64
Explanation:
0 → Meme has engagement BELOW the median (Not trending)
1 → Meme has engagement ABOVE the median (Trending)
```

Step 2 : Encoding Categorical Data

Next, meme engagement data is prepared for machine learning by encoding categorical variables in the Dataframe and selecting key features for model training.

Next, the code defines the features (inputs) that will be used for training a machine learning model. It selects the columns 'Likes', 'Shares', 'Comments', 'Sentiment Score', 'Content Type', and 'Engagement Level' as the features, while the target variable for classification is set to 'Trend'. A dictionary called **label_encoders** is created to store the label encoder objects for each of these columns. The code then iterates through each specified column, creating a LabelEncoder instance for each one. This encoder transforms the categorical values into numeric labels, which are more suitable for the machine learning algorithms. After encoding each column, the code prints out the classes that were encoded, allowing for verification of the transformation.

This process is essential for preparing the data for analysis, as machine learning models require numerical input. By encoding categorical variables and clearly defining features and targets, the code sets the stage for effective model training and evaluation. Overall, it defines a set of features, likes, shares, comments, sentiment score, content type, and engagement level as inputs (X) for model training, with the Trend label serving as the target variable (y) for classification.

In [17]:
```
# encodes categorical variables
label_cols = ['Sentiment Score', 'Content Type', 'Engagement Level']  # columns to encode
label_encoders = {}  # dictionary to store label encoder objects
for col in label_cols:
    le = LabelEncoder()  # creates a LabelEncoder instance
    df_memes[col] = le.fit_transform(df_memes[col])  # encodes categorical column to numeric labels
    label_encoders[col] = le  # saves encoder for possible inverse transform
    print(f"Encoded '{col}' with classes: {list(le.classes_)}")  # prints encoded classes for verification

# define features (inputs) for training
features = ['Likes', 'Shares', 'Comments', 'Sentiment Score', 'Content Type', 'Engagement Level']  # list of feature column names
X = df_memes[features]  # selects feature columns from dataframe for model input
y = df_memes['Trend']  # selects target variable column for classification

print(f"Features selected: {features}")  # confirms selected features
print(f"Target variable set: 'Trend' with {y.nunique()} classes")  # confirms number of classes in target
print(f"Feature sample:\n{X.head()}")  # shows sample of feature data
print(f"Target sample:\n{y.head()}")  # shows sample of target labels
```

```
Encoded 'Sentiment Score' with classes: ['Negative', 'Neutral', 'Positive']
Encoded 'Content Type' with classes: ['Image', 'Shorts']
Encoded 'Engagement Level' with classes: ['High', 'Low', 'Medium']
Features selected: ['Likes', 'Shares', 'Comments', 'Sentiment Score', 'Content Type', 'Engagement Level']
Target variable set: 'Trend' with 2 classes
Feature sample:
   Likes  Shares  Comments  Sentiment Score  Content Type  Engagement Level
0   3394    1081       275                1             0                 1
1   7777    1690       450                1             1                 1
2   6106    2997       222                1             0                 1
3   9612      88        38                1             0                 1
4   3733     400       736                0             0                 1
Target sample:
0    0
1    1
2    0
3    1
4    1
Name: Trend, dtype: int64
```

In [18]:
```
df_memes.shape
```

Out[18]: (16628, 16)

Step 3: Standardising the Data for the Machine Learning Model

This code standardises the feature values to ensure they are on a consistent scale, which helps improve the performance of machine learning models.

It uses StandardScaler, a preprocessing tool that transforms the data by subtracting the mean and dividing by the standard deviation.

This ensures that all features have a mean of 0 and a standard deviation of 1, preventing any single feature from dominating the model due to differences in scale. The transformed dataset (X_scaled) is then ready for use in model training.

In [19]:
```
# standardises feature values to ensure consistent scale across variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# prints the verification information
print(f"Original data shape: {X.shape}")
print(f"Scaled data shape: {X_scaled.shape}")
print("First 5 rows of scaled data:")
print(X_scaled[:5])
```

```
Original data shape: (16628, 6)
Scaled data shape: (16628, 6)
First 5 rows of scaled data:
[[-0.5008004  -0.26887966 -0.34790521 -0.50708505 -0.313385   -0.27122968]
 [-0.15525652  0.09635769  0.02298652 -0.50708505  3.1909632  -0.27122968]
 [-0.28699364  0.88020861 -0.46023242 -0.50708505 -0.313385   -0.27122968]
 [-0.01059007 -0.86441444 -0.85019858 -0.50708505 -0.313385   -0.27122968]
 [-0.47407456 -0.67729777  0.62912957 -2.03569457 -0.313385   -0.27122968]]
```

Step 4: Dataset Splitting

The dataset is split into training and testing sets to prepare for machine learning. It assigns 80% of the data for training (X_train, y_train) and 20% for testing (X_test, y_test) using train_test_split. The random_state parameter ensures consistency in the split for reproducibility. After splitting, it prints the sizes of both sets, helping verify the distribution before proceeding with model training.

In [20]:
```
#split datasets into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

#displays the training and testing sample size
print("Training Set Size:", X_train.shape)
print("Testing Set Size:", X_test.shape)
```

```
Training Set Size: (13302, 6)
Testing Set Size: (3326, 6)
```

Step 5: Establish Baseline Accuracy

Before applying advanced classifiers, it's essential to establish a baseline for comparison.

A baseline model is established for classification by predicting the majority class from the training data. It first determines which class appears most frequently in y_train, then uses that class as the predicted label for all test samples (y_test). By comparing these predictions to the actual test labels, it calculates the baseline accuracy, which serves as a reference point for evaluating more advanced models. The final output prints the total number of test samples, correct predictions, and the baseline accuracy as a percentage, helping assess whether a more sophisticated model would provide meaningful improvements.

As resulted below, the baseline model achieved an accuracy of **49.40%** by predicting the majority class from the training data. This result indicates that if one were to simply predict the most frequently occurring class for all test samples, approximately 50% of them would be correctly classified. This baseline accuracy serves as a reference point for evaluating the performance of more advanced classifiers which will be conducted below. If a sophisticated model does not outperform this baseline, it may suggest that the model is not effectively capturing the underlying patterns in the data or that the features used are not informative enough. Furthermore, the baseline model highlights the importance of understanding class distribution within the dataset. A high baseline accuracy reflects an imbalanced dataset where one class significantly outnumbers others, emphasising the need for careful consideration of class balance when interpreting results.

The baseline model's accuracy thus provides a foundational understanding of the classification task, allowing for meaningful comparisons with more advanced models below and ensuring that any improvements are both significant and thus justifiable.

In [21]:
```
#determines the majority class in the training data
majority_class = Counter(y_train).most_common(1)[0][0]
print(f"Majority Class in Training Data: {majority_class}")

#creates baseline predictions (all predictions as majority class)
baseline_preds = [majority_class] * len(y_test)
print(f"Sample Baseline Predictions: {baseline_preds[:10]}")  # displays the first 10 predictions

#calculates the baseline accuracy
baseline_accuracy = accuracy_score(y_test, baseline_preds)
print(f"Total Test Samples: {len(y_test)}")
print(f"Correct Predictions (Baseline): {sum(y_test == majority_class)}")

#prints baseline accuracy as a percentage
print(f"Baseline Accuracy: {baseline_accuracy:.2%}")
```

```
Majority Class in Training Data: 0
Sample Baseline Predictions: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Total Test Samples: 3326
Correct Predictions (Baseline): 1643
Baseline Accuracy: 49.40%
```

Step 6: Train Logistic Regression Model

Why is LR Chosen?

Logistic Regression (LR) was chosen for this project for several reasons. Firstly, the primary task involves predicting a binary outcome, specifically the 'Trend' of meme posts, which can be classified into two categories (e.g., trending or not trending). Logistic regression is inherently designed for binary classification problems, making it a suitable choice. Additionally, one of the key advantages of logistic regression is its interpretability. The model provides coefficients for each feature, indicating the strength and direction of the relationship between the features and the target variable. This transparency is valuable for understanding how different factors, such as likes, shares, and sentiment scores, influence the likelihood of a post trending.

Moreover, logistic regression is computationally efficient and can handle large datasets with ease. Given that meme-related data can be extensive, using a model that is quick to train and predict is advantageous, especially when iterating through different feature sets or tuning hyperparameters. Another significant benefit is that logistic regression not only predicts class labels but also provides probabilities for each class. This feature allows for a more nuanced understanding of predictions, enabling the identification of posts that are not just predicted to trend but also have a high probability of doing so, which can be particularly useful for decision-making processes

The LR model is first initialised with a specified random state of 42 for reproducibility. The model is then trained using the training data, represented by X_train and y_train. After training, the model generates predictions on the test data (X_test) and calculates the predicted probabilities for the positive class.

Next, the code calculates the overall accuracy of the model by comparing the predicted labels (y_pred) with the actual labels from the test set (y_test). This accuracy is then printed prominently, providing a clear indication of the model's performance.

To further evaluate the model, various metrics are calculated and stored in a dictionary called lrmodel_metrics. These metrics include overall accuracy, the area under the ROC curve (AUC-ROC), the R^2 score, and the mean squared error (MSE). The R^2 score and MSE are calculated separately for clarity.

The metrics are then converted into a pandas DataFrame for better readability, rounding the values to two decimal places. Additionally, a classification report is generated, which provides detailed statistics such as precision, recall, and F1-score for each class. This report is also transformed into a DataFrame and adjusted to display percentages for the relevant metrics.

Finally, a confusion matrix is created, which visually represents the model's performance in terms of true positives, true negatives, false positives, and false negatives. This matrix is displayed using a heatmap for easy interpretation.

The Logistic Regression acheived an accuracy of **72.16%.**

In [22]:
```
# intialises and trains the model
lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train, y_train)

# generates predictions and probabilities
y_pred = lr_model.predict(X_test)
y_proba = lr_model.predict_proba(X_test)[:, 1]

# calculates the overall accuracy
```

```
overall_accuracy = accuracy_score(y_test, y_pred) * 100

print("\n" + "="*50)
print(f"LOGISTIC REGRESSION MODEL ACCURACY: {overall_accuracy:.2f}%")
print("="*50)

# calculates the metrics including R² and MSE on predicted probabilities
lrmodel_metrics = {
    'Accuracy': overall_accuracy,
    'AUC-ROC': roc_auc_score(y_test, y_proba) * 100,
    'R² SCORE': r2_score(y_test, y_proba),
    'MSE': mean_squared_error(y_test, y_proba)
}

# creates metrics DataFrame
lrmodel_metrics_df = pd.DataFrame(list(lrmodel_metrics.items()), columns=['Metric', 'Value'])
lrmodel_metrics_df['Value'] = lrmodel_metrics_df['Value'].astype(float).round(4)

# generates classification report
lrmodel_class_report = classification_report(y_test, y_pred, output_dict=True)
lrmodel_class_report_df = pd.DataFrame(lrmodel_class_report).transpose()

# formats the precision, recall, and f1-score as percentages
for col in ['precision', 'recall', 'f1-score']:
    if col in lrmodel_class_report_df.columns:
        lrmodel_class_report_df[col] = (lrmodel_class_report_df[col] * 100).round(2)

# prints the evaluation results
print("\nDETAILED EVALUATION METRICS:")
print(lrmodel_metrics_df.to_string(index=False))

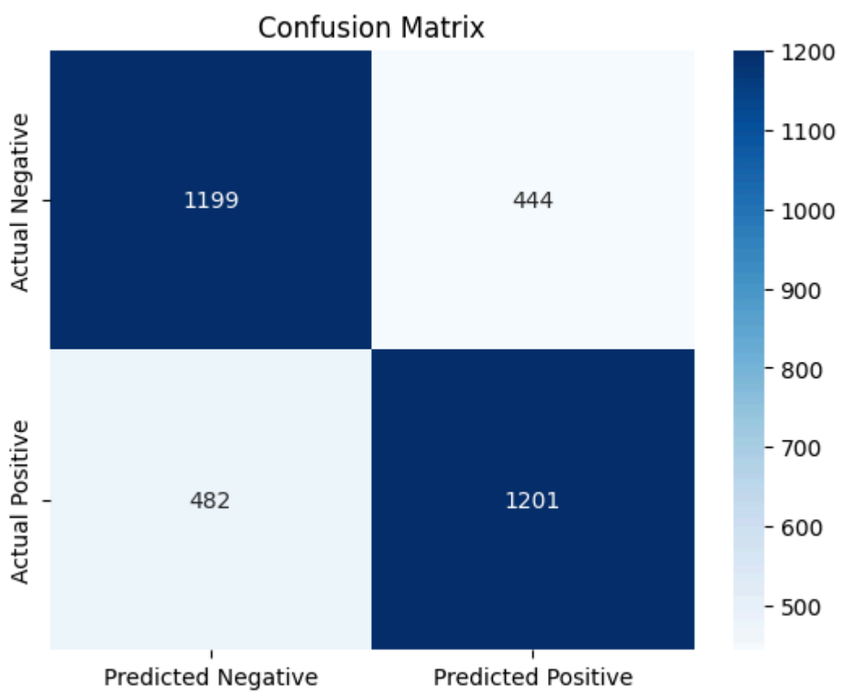
print("\nCLASSIFICATION REPORT:")
print(lrmodel_class_report_df.to_string())

# creates and plots confusion matrix
cm = confusion_matrix(y_test, y_pred)
cm_df = pd.DataFrame(cm,
                     index=['Actual Negative', 'Actual Positive'],
                     columns=['Predicted Negative', 'Predicted Positive'])
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

```
==================================================
LOGISTIC REGRESSION MODEL ACCURACY: 72.16%
==================================================

DETAILED EVALUATION METRICS:
  Metric   Value
Accuracy 72.1587
 AUC-ROC 81.0392
R² SCORE  0.2835
     MSE  0.1791

CLASSIFICATION REPORT:
              precision  recall  f1-score      support
0                 71.33   72.98     72.14  1643.000000
1                 73.01   71.36     72.18  1683.000000
accuracy          72.16   72.16     72.16     0.721587
macro avg         72.17   72.17     72.16  3326.000000
weighted avg      72.18   72.16     72.16  3326.000000
```



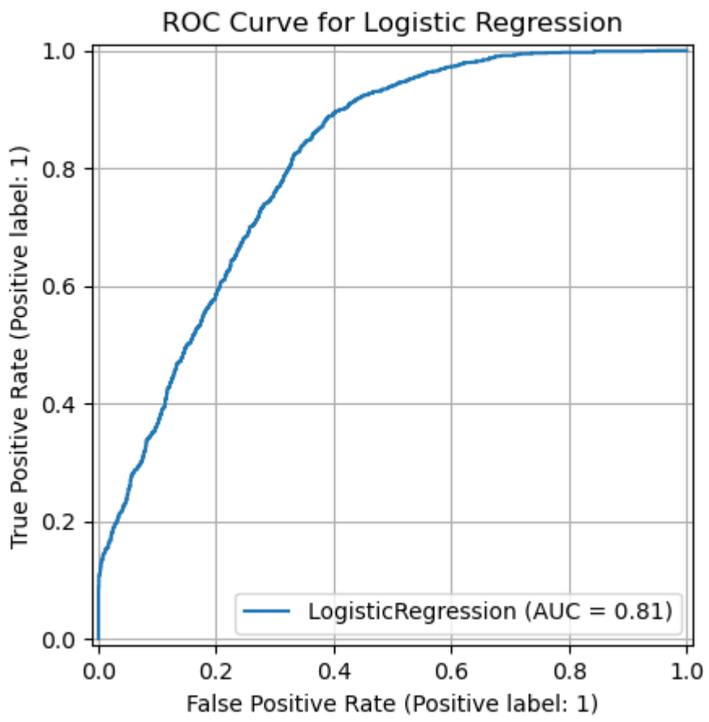
The ROC curve for the Logistic Regression model rises clearly above the diagonal baseline, with an AUC of 81.00%, indicating strong discriminative ability between trending and non-trending memes. Its shape suggests the model is responsive to threshold tuning for optimising for precision at the expense of recall. This precision-oriented behaviour makes it ideal for scenarios where false positives are costly, such as targeted marketing, though it may miss a portion of genuinely viral content.

In [33]:
```
# LR Model ROC Curve
plt.figure(figsize=(10, 6))  # sets figure size to 10x6 inches for better visibility

# generates and displays ROC Curve for Logistic Regression model
# X_test: Test dataset features
# y_test: True labels for test dataset
RocCurveDisplay.from_estimator(lr_model, X_test, y_test)

plt.title('ROC Curve for Logistic Regression')  # adds descriptive title
plt.grid(True)  # allows grid lines for better readability
plt.show()  # displays the plot
```

```
<Figure size 1000x600 with 0 Axes>
```



Step 7: Train Random Forest Model

Why was the Random Forest Model Chosen?

The Random Forest model was chosen for its high accuracy and robustness in handling complex datasets, making it ideal for various applications. Its ability to reduce overfitting through ensemble learning, along with effective variable importance assessment, enhances its performance in predictive tasks.

Additionally, the Random Forest model excels in managing both categorical and continuous variables, providing versatility across different types of data. Its inherent capability to handle missing values without significant loss of performance further solidifies its suitability for real world applications. The model's parallel processing nature allows for faster training times, making it efficient for large datasets. Moreover, the ensemble approach of combining multiple decision trees not only improves prediction accuracy but also enhances the model's stability, ensuring reliable results even in the presence of noise and outliers. This combination of features makes Random Forest a preferred choice for many data scientists and machine learning practitioners.

The Random Forest Model is first created using an instance of the RandomForestClassifier, setting a random state of 42 for reproducibility. The model is then trained using the training data, represented by X_train and y_train. After training, the model generates predictions on the test data (X_test) and calculates the predicted probabilities for the positive class.

Following this, the code calculates the overall accuracy of the model by comparing the predicted labels (rf_preds) with the actual labels from the test set (y_test). This accuracy is printed prominently, providing a clear indication of the model's performance.

To further evaluate the model, various metrics are calculated and stored in a dictionary called rf_model_metrics. These metrics include overall accuracy and the area under the ROC curve (AUC-ROC). The metrics are then converted into a pandas DataFrame for better readability, rounding the values to two decimal places.

Additionally, a classification report is generated, which provides detailed statistics such as precision, recall, and F1-score for each class. This report is transformed into a DataFrame and adjusted to display percentages for the relevant metrics. The code then prints the detailed evaluation results, including both the metrics and the classification report.

Finally, a confusion matrix is created to visually represent the model's performance in terms of true positives, true negatives, false positives, and false negatives. This matrix is displayed using a heatmap for easy interpretation.

The Random Forest Model acheived an accuracy of **75.59%**.

In [32]:
```
# initialises and trains the Random Forest model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# generates the predictions and probabilities
rf_preds = rf_model.predict(X_test)
rf_probs = rf_model.predict_proba(X_test)[:, 1]

# calculates the overall accuracy first
overall_accuracy = accuracy_score(y_test, rf_preds) * 100

# prints the overall accuracy prominently at the top
print("\n" + "="*50)
print(f"RANDOM FOREST MODEL ACCURACY: {overall_accuracy:.2f}%")
print("="*50)

# calculates the metrics of the model for evaluation
rf_model_metrics = {
    'Accuracy': overall_accuracy,
    'AUC-ROC': roc_auc_score(y_test, rf_probs) * 100,
    'R² Score': r2_score(y_test, rf_probs),  # R² Score
    'MSE': mean_squared_error(y_test, rf_probs)  # Mean Squared Error
}

# creates the metrics DataFrame
rf_model_metrics_df = pd.DataFrame(list(rf_model_metrics.items()), columns=['Metric', 'Value'])
rf_model_metrics_df['Value'] = rf_model_metrics_df['Value'].astype(float).round(4)


# generates and prints the classification report
rf_model_class_report = classification_report(y_test, rf_preds, output_dict=True)
rf_model_class_report_df = pd.DataFrame(rf_model_class_report).transpose()
rf_model_class_report_df = rf_model_class_report_df.apply(lambda x: x * 100 if x.name in ['precision', 'recall', 'f1-score'] else x)
rf_model_class_report_df = rf_model_class_report_df.round(2)

# prints the detailed evaluation results
print("\nDETAILED EVALUATION METRICS:")
print(rf_model_metrics_df.to_string(index=False))

print("\nCLASSIFICATION REPORT:")
print(rf_model_class_report_df.to_string())

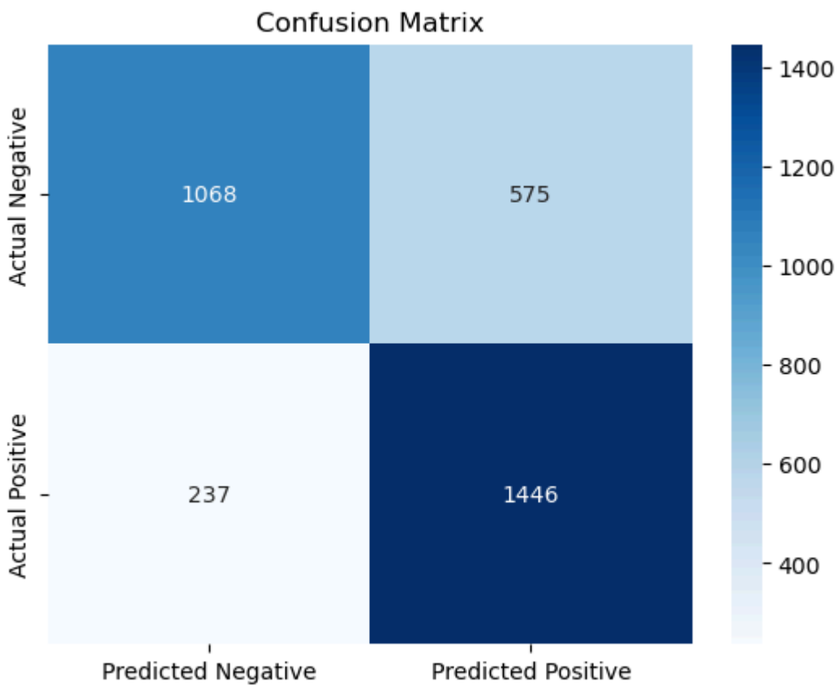
# creates the confusion matrix
cm = confusion_matrix(y_test, rf_preds)
cm_df = pd.DataFrame(cm,
                     index=['Actual Negative', 'Actual Positive'],
                     columns=['Predicted Negative', 'Predicted Positive'])

# shows the confusion matrix
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

```
==================================================
RANDOM FOREST MODEL ACCURACY: 75.59%
==================================================

DETAILED EVALUATION METRICS:
  Metric   Value
Accuracy 75.5863
 AUC-ROC 83.4438
R² Score  0.3997
     MSE  0.1501

CLASSIFICATION REPORT:
              precision  recall  f1-score  support
0                 81.84   65.00     72.46  1643.00
1                 71.55   85.92     78.08  1683.00
accuracy          75.59   75.59     75.59     0.76
macro avg         76.69   75.46     75.27  3326.00
weighted avg      76.63   75.59     75.30  3326.00
```



Receiver Operating Characteristic (ROC) Curve Analysis

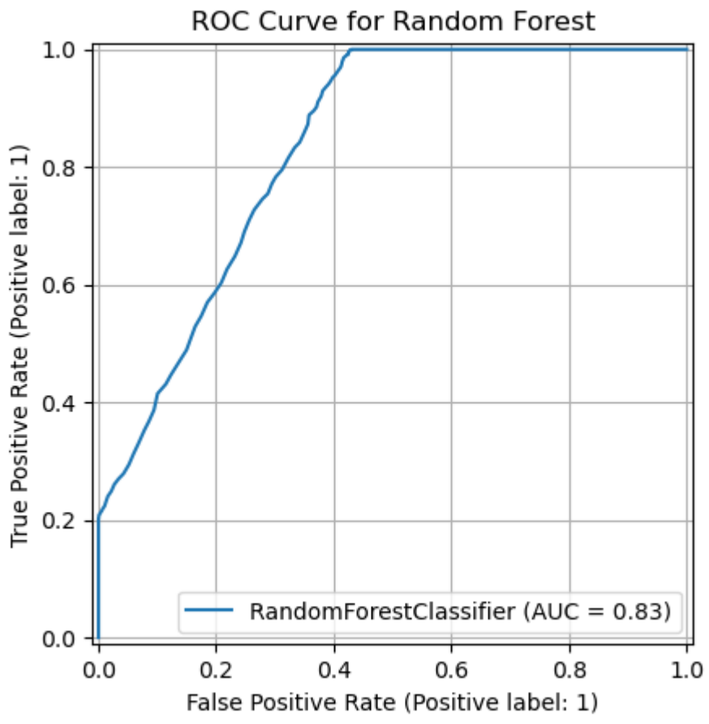
The Random Forest model's ROC curve shows a pronounced arc above the baseline, with an AUC of 83.00%, reflecting strong discriminative power and a well-balanced trade-off between sensitivity and specificity. Its shape indicates robustness across threshold variations, making it ideal for exploratory trend detection where capturing a broad spectrum of viral content is key even if it means accepting some false positives. This performance stems from Random Forest's ability to model non-linear relationships and complex feature interactions, which is evident in the curve's superior form.

In [44]:
```
# RF Model ROC Curve
plt.figure(figsize=(10, 6))  # sets figure size to 10x6 inches for better visibility

# generates and displays ROC Curve for Random Forest model
# X_test: Test dataset features
# y_test: True labels for test dataset
RocCurveDisplay.from_estimator(rf_model, X_test, y_test)

plt.title('ROC Curve for Random Forest')  # adds descriptive title
plt.grid(True)  # allows grid lines for better readability
plt.show()  # displays the plot
```

```
<Figure size 1000x600 with 0 Axes>
```



Feature Importance (Random Forest)

Feature importance analysis from the Random Forest model provides valuable insight into the relative contribution of each variable to meme virality prediction. According to the model, Shares emerged as the most influential feature, followed by Likes and Comments, highlighting the dominant role of direct engagement metrics in driving predictive performance. These features reflect visible user interactions that are often publicly accessible and quantifiable, making them powerful indicators of a meme's viral potential. Engagement Level, a composite measure of overall interaction, also showed moderate importance, reinforcing the value of capturing broader audience activity patterns rather than relying on isolated metrics.

In contrast, Sentiment Score and Content Type were found to have the lowest importance scores, suggesting that while the emotional tone and structural format of a meme may contribute to its appeal, they were less critical in this dataset compared to tangible user engagement. This finding is particularly relevant for marketers and content strategists, as it implies that efforts to increase visibility such as encouraging shares and likes may yield stronger predictive signals for virality than adjusting sentiment or format alone. By prioritising features with higher predictive weight, practitioners can more effectively tailor content to maximise reach and resonance in fast-moving digital environments.

In [48]:
```
# extracts the feature importance scores from the trained Random Forest model
# the scores represents each feature's contribution to prediction accuracy
feature_importances = pd.Series(rf_model.feature_importances_, index=features)

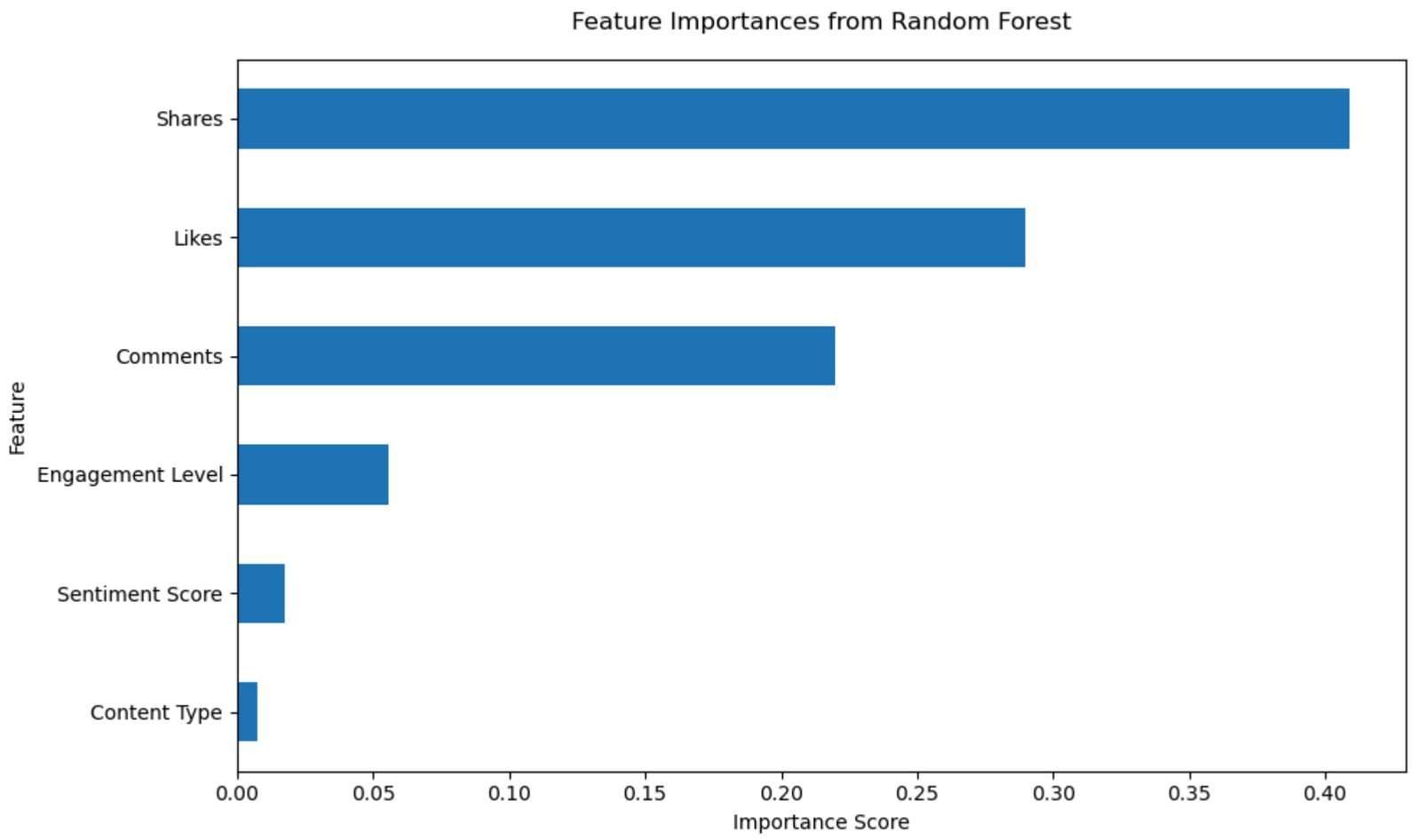
# sorts the feature importances in ascending order and plot as horizontal bar chart
feature_importances.sort_values().plot(
    kind='barh',  # sets the horizontal bar plot for better readability
    figsize=(10,6)  # sets the custom figure size
)

# adds a descriptive title to contextualise the visualisation
plt.title('Feature Importances from Random Forest', pad=15)  # adds spacing

# adds the x-axis label to clarify the metric being shown
plt.xlabel('Importance Score')  # sets the scores range from 0 (no importance) to 1 (max importance)

# adds the y-axis label to identify the features
plt.ylabel('Feature')
```

```
# adjusts the layout and display the plot
plt.tight_layout()  # prevents label cutoff
plt.show()
```



Cross Validation To Evaluate Model Robustness

Next, cross validation is implemented to evaluate the performance of the two classification machine learning models, **Logistic Regression** and **Random Forest**. Cross-validation is a statistical method used to assess how the results of a model will generalise to the dataset.

Firstly, conducting cross validation helps to mitigate the risk of overfitting, which occurs when a model learns the noise in the training data rather than the underlying patterns. By splitting the dataset into multiple subsets (or folds), the model is trained on a portion of the data and validated on another, ensuring that it is tested on unseen data. This process provides a more reliable estimate of the model's performance compared to a single train-test split.

In this code, the **StratifiedKFold** method is used, which ensures that each fold maintains the same proportion of classes as the entire dataset. This is particularly important for imbalanced datasets, where some classes may be underrepresented. By preserving the class distribution, the model's evaluation becomes more robust and reflective of its true performance across different classes.

The code calculates cross-validation scores based on accuracy for both models, providing a set of scores that reflect how well each model performs across the different folds. The mean accuracy score gives an overall indication of the model's performance, while the standard deviation provides insight into the variability of the model's performance across the folds. A low standard deviation indicates that the model's performance is consistent, while a high standard deviation may suggest that the model's performance is sensitive to the specific data used in training and testing.

In summary, cross-validation is a crucial technique in machine learning that enhances model evaluation by providing a more accurate estimate of performance, reducing the risk of overfitting, and ensuring that the model generalises well to unseen data. By employing cross-validation, more informed decisions about model selection and tuning can be made for future development, ultimately leading to better predictive performance in real-world applications.

In [52]:
```
# defines the cross-validation strategy
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# prints the Logistic Regression Model Cross Validation
print("\n" + "="*50)
print("Cross Validation: Logistic Regression Model")
print("="*50)

# calculates the cross-validation scores based on accuracy
lr_cv_scores = cross_val_score(lr_model, X_scaled, y, cv=cv_strategy, scoring='accuracy')

# prints the results
print(f"Cross Validation Scores (Accuracy): {lr_cv_scores}")
print(f"Mean CV Accuracy: {np.mean(lr_cv_scores):.4f}")
print(f"Standard Deviation: {np.std(lr_cv_scores):.4f}")

# prints the Random Forest Model Cross Validation
print("\n" + "="*50)
print("Cross Validation: Random Forest Model")
print("="*50)

# calculates the cross-validation scores based on accuracy
rf_cv_scores = cross_val_score(rf_model, X_scaled, y, cv=cv_strategy, scoring='accuracy')

# prints the results
print(f"Cross Validation Scores (Accuracy): {rf_cv_scores}")
print(f"Mean CV Accuracy: {np.mean(rf_cv_scores):.4f}")
print(f"Standard Deviation: {np.std(rf_cv_scores):.4f}")
```

```
==================================================
Cross Validation: Logistic Regression Model
==================================================
Cross Validation Scores (Accuracy): [0.70805773 0.70625376 0.71677691 0.72601504 0.71218045]
Mean CV Accuracy: 0.7139
Standard Deviation: 0.0071

==================================================
Cross Validation: Random Forest Model
==================================================
Cross Validation Scores (Accuracy): [0.75706554 0.76578473 0.76398076 0.75338346 0.75909774]
Mean CV Accuracy: 0.7599
Standard Deviation: 0.0045
```

The cross-validation results for the Logistic Regression model indicate a consistent performance across the five folds. The accuracy scores for each fold were as follows: 0.7081, 0.7063, 0.7168, 0.7260, and 0.7122. The mean cross-validation accuracy for the Logistic Regression model was calculated to be 0.7139, with a standard deviation of 0.0071. This low standard deviation suggests that the model's performance is stable and reliable across different subsets of the data.

In contrast, the Random Forest model demonstrated a higher level of accuracy in its cross-validation results. The accuracy scores for the five folds were 0.7571, 0.7658, 0.7640, 0.7534, and 0.7591. The mean cross-validation accuracy for the Random Forest model was 0.7599, with a standard deviation of 0.0045, indicating even greater consistency in its performance compared to the Logistic Regression model.

Overall, the Random Forest model outperformed the Logistic Regression model in terms of mean accuracy, suggesting it may be a more effective choice for the classification task at hand. Both models exhibited low standard deviations, indicating their reliability, but the Random Forest's higher mean accuracy highlights its superior capability in this context.

LSTM Model

The **attention-based LSTM (Long Short-Term Memory) model** is specifically designed for time-series forecasting and sequential data analysis, making it highly effective for predicting meme trends based on engagement metrics. Before training, the dataset undergoes several key preprocessing steps:

- Timestamp Conversion: Captures time-based trends, enabling the model to detect evolving patterns in engagement.

- Categorical Encoding: Transforms non-numeric features (e.g., sentiment categories, content types) into numerical formats for model compatibility.
- Normalisation: Standardises numerical features like likes, shares, and comments to ensure balanced input scales.
- Data Splitting: Divides the dataset into training, validation, and testing subsets to rigorously evaluate model performance.

The attention-based LSTM was chosen for meme trend prediction due to its advanced ability to capture temporal dependencies and contextual relevance in sequential data. Memes often exhibit time-sensitive dynamics influenced by cultural shifts, current events, and audience sentiment. While traditional LSTMs are adept at handling long-term dependencies, the attention mechanism enhances this by allowing the model to selectively focus on the most informative time steps—prioritising key engagement signals while filtering out noise.

This architectural enhancement enables the model to learn complex relationships across time, making it particularly suited for identifying nuanced patterns in likes, shares, and comments. By attending to critical moments in a meme's engagement trajectory, the model can make more informed and confident predictions about virality.

Once the data is preprocessed, the attention-based LSTM is trained on sequential engagement metrics. It evaluates predictions using metrics such as accuracy and AUC-ROC, adjusting its learning based on validation feedback. This approach allows the model to capture both short-term spikes and long-term trends in meme engagement, resulting in more reliable predictions.

The attention-based LSTM achieved an accuracy of **78.26%**, outperforming other models and demonstrating its strength in identifying trending content through temporal and contextual analysis.

Step 1 : Define Attention Layer & Features & Target

In [34]:
```
# loads dataset
LSTM_df_memes = pd.read_csv("cleaned_datasets/combined_dataset.csv")
```

Defines Attention Layer

The Attention layer is designed to help the model focus on the most important parts of the input sequence. It starts by creating trainable weights and biases that are used to calculate attention scores for each timestep in the sequence. These scores are computed using a tanh activation function, which introduces non-linearity. The scores are then normalised with a softmax function to produce attention weights, representing how much focus each timestep should receive. These weights are applied to the input data through element-wise multiplication, creating a weighted context vector. Finally, this context vector is summed across all timesteps to produce a single vector that emphasises the most relevant information in the sequence. This mechanism allows the model to better capture key temporal patterns by selectively attending to important moments.

In [38]:
```
# defines the Attention Layer
class Attention(Layer):
    def __init__(self, **kwargs):
        # intialises the parent Layer class
        super(Attention, self).__init__(**kwargs)

    def build(self, input_shape):
        # creates trainable weight matrix for attention
        self.W = self.add_weight(name="att_weight",
                                 shape=(input_shape[-1], 1),  # shapes based on input features
                                 initializer="random_normal",  # intialises weights randomly
                                 trainable=True)  # makes the weights trainable

        # creates trainable bias vector for attention
        self.b = self.add_weight(name="att_bias",
                                 shape=(input_shape[1], 1),  # shapes based on input sequence length
                                 initializer="zeros",  # intialises biases to zero
                                 trainable=True)  # makes the biases trainable

        # calls the parent build method
        super(Attention, self).build(input_shape)

    def call(self, inputs):
        # computes the attention scores using a tanh activation
        e = tf.keras.backend.tanh(tf.keras.backend.dot(inputs, self.W) + self.b)

        # squeezes the last dimension to get a 1D tensor of scores
        e = tf.keras.backend.squeeze(e, axis=-1)

        # applies softmax to get attention weights (alpha)
        alpha = tf.keras.backend.softmax(e)

        # explands the dimensions to match the input shape for element-wise multiplication
        alpha = tf.keras.backend.expand_dims(alpha, axis=-1)

        # computes the context vector as a weighted sum of the inputs
        context = inputs * alpha

        # sums the context vector across the time steps
        context = tf.keras.backend.sum(context, axis=1)

        # returns the context vector
        return context
```

Firstly, the meme dataset is prepared once again for machine learning by performing several preprocessing steps. It first converts the 'Date' column to a datetime format, ensuring proper sorting and manipulation of time-based data. Next, it calculates a 'Trend' label based on engagement metrics, marking memes in the top 50% of total engagement as trending.

It then transforms categorical data into numerical values. Sentiment labels (Positive, Neutral, Negative) are mapped to numeric scores (-1, 0, 1), while engagement levels (Low, Medium, High) are encoded as 0, 1, and 2, respectively. This ensures the data is structured correctly for model training.

Finally, key features including Likes, Shares, Comments, Sentiment Score, and Engagement Level are selected for analysis. To standardise the data, StandardScaler is applied, normalising feature values to improve model performance. This process helps ensure that all features are on a consistent scale, making them more suitable for machine learning algorithms.

In [41]:
```
#converts the 'Date' column to datetime format for proper sorting and manipulation
LSTM_df_memes['Date'] = pd.to_datetime(LSTM_df_memes['Date'], errors='coerce')

#defines the trend status based on engagement metrics (Views, Likes, Shares, Comments)
LSTM_df_memes['Trend'] = ((LSTM_df_memes['Views'] + LSTM_df_memes['Likes'] + LSTM_df_memes['Shares'] + LSTM_df_memes['Comments']) >
                          LSTM_df_memes[['Views', 'Likes', 'Shares', 'Comments']].sum(axis=1).quantile(0.5)).astype(int)

#converts sentiment labels ('Positive', 'Neutral', 'Negative') into numerical values
label_map = {"Negative": -1, "Neutral": 0, "Positive": 1}
LSTM_df_memes['Sentiment Score'] = LSTM_df_memes['Sentiment Score'].map(label_map)

#encodes engagement levels ('Low', 'Medium', 'High') into numerical values
engagement_map = {"Low": 0, "Medium": 1, "High": 2}
LSTM_df_memes['Engagement Level'] = LSTM_df_memes['Engagement Level'].map(engagement_map)

#selects the key features for the model
features = ['Likes', 'Shares', 'Comments', 'Sentiment Score', 'Engagement Level']

#normalises feature values to improve model performance
scaler = StandardScaler()
LSTM_df_memes[features] = scaler.fit_transform(LSTM_df_memes[features])
```

Step 2 : Creates Sequences for LSTM Input

Next, the data is prepared for time series analysis using the LSTM model. A function, **create_sequences()** is defined, which generates input sequences and their corresponding labels. The function takes a dataset, a target variable, and a sequence length (seq_length) as inputs, then iterates through the data to extract sequential subsets for training.

For each sequence:

- X contains a rolling window of seq_length previous data points.
- y holds the trend label corresponding to the next data point after the sequence.

By structuring data this way, the LSTM model can learn temporal patterns in meme engagement, helping predict whether future memes will become trending based on past engagement. The function returns NumPy arrays, ensuring efficient handling of sequence data during training. Finally, the script applies the function to the dataset, generating input (X) and target (y) values for model training.

In [45]:
```
#function to generate sequences of data for time series analysis
def create_sequences(data, target, seq_length=10):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])  #collects the sequences of input data
        y.append(target[i+seq_length])  #stores the corresponding labels
    return np.array(X), np.array(y)

#defines the sequence length for LSTM
seq_length = 10

#generates sequences from the dataset
X, y = create_sequences(LSTM_df_memes[features].values, LSTM_df_memes['Trend'].values, seq_length)
```

Step 3 : Dataset Splitting into Holdout Validation Sets

This code divides the dataset into three sets to ensure effective model training and evaluation. First, it allocates 70% of the data for training (X_train, y_train), where the model learns patterns from past meme engagement. The remaining 30% is temporarily stored (X_temp, y_temp) before being further split into validation and testing sets.

- Validation Set (15%): Used during training to fine-tune the model and prevent overfitting.
- Testing Set (15%): Used after training to assess final model performance on unseen data.

The code prints the sizes of each dataset to confirm the split, ensuring the distribution supports accurate training and evaluation.

In [49]:
```
#splits the dataset into training (70%), validation (15%), and testing (15%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)  #keeps 70% for training
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.2, random_state=42)  #splits the remaining 30%

#prints dataset split sizes
print(f"Training Set Size: {X_train.shape[0]} samples")
print(f"Validation Set Size: {X_val.shape[0]} samples")
print(f"Testing Set Size: {X_test.shape[0]} samples")
```

```
Training Set Size: 11632 samples
Validation Set Size: 3988 samples
Testing Set Size: 998 samples
```

Step 4 : Builds ands Configures the LSTM Baseline Model

Next, the LSTM (Long Short-Term Memory) model is defined and designed for binary classification, specifically to predict whether a meme will trend based on engagement features. The architecture consists of:

- **Two Bidirectional LSTM layers**, first layer with 128 units and ReLU activation and second layer with 64 units and ReLu activation. The bidirectional configuration allows the model to learn sequential patterns in engagement data from both past and future contexts, enhancing its ability to capture temporal dependencies.
- Return sequences is enabled in the first LSTM layer to allow the output to be passed to the next layer, facilitating the learning of complex patterns over time.
- A **third LSTM layer** with 32 units (not bidirectional to reduce complexity), which further refines the learned representations before making predictions.
- An **Attention layer** to further enhance the model's capability to focus on the most relevant parts of the engagement sequence, an attention mechanism is incorporated following the last LSTM layer. This attention layer computes a weighted context vector by learning to assign different importance to each timestep's output from the LSTM, allowing the model to selectively concentrate on key moments in the sequence that contribute most strongly to predicting meme virality. By doing so, the model can better capture temporal dependencies and nuanced patterns in the engagement data, improving interpretability and potentially boosting predictive performance beyond standard LSTM architectures.
- A **dense hidden layer** with 32 units and ReLU activation, followed by **dropout layers (with a dropout rate of 0.3)** to prevent overfitting by randomly setting a fraction of the input units to zero during training.
- The **output layer** is a dense layer with a sigmoid activation function, ensuring that the final prediction is a probability value between 0 and 1, suitable for the binary classification tasks.

Once the model structure is defined, it is compiled using:

- The **Adam optimiser**, which efficiently adjusts learning rates during training, helping to converge to optimal solutions more quickly.
- **Binary crossentropy loss**, which is appropriate for binary classification tasks, measuring the performance of the model by comparing the predicted probabilities to the actual labels.
- **Accuracy** as the metric to provide a straightforward way to evaluate the model's performance during training and validation.

Finally, the model summary is printed, displaying the layer details and parameter counts, confirming the setup before training. This approach enables effective trend forecasting based on historical engagement data.

In [55]:
```
# initalises the Sequential model
LSTM_Model = Sequential([
    # adds the First Bidirectional LSTM layer with 128 units, using ReLU activation and returning sequences
    Bidirectional(LSTM(128, activation='relu', return_sequences=True), input_shape=(seq_length, X_train.shape[2])),
    Dropout(0.3),  # adds Dropout layer to prevent overfitting

    # adds the Second Bidirectional LSTM layer with 64 units, returning sequences
    Bidirectional(LSTM(64, activation='relu', return_sequences=True)),
    Dropout(0.3),  # adds another Dropout layer

    # adds the Third LSTM layer with 32 units, returning sequences for attention mechanism
    LSTM(32, activation='relu', return_sequences=True),
    Dropout(0.3),  # Dropout layer

    # adds the Attention layer to focus on important parts of the input sequence
    Attention(),

    # adds the Fully connected layer with 32 units and ReLU activation
    Dense(32, activation='relu'),
    Dropout(0.3),  # Dropout layer

    # adds the final Output layer with 1 unit and sigmoid activation for binary classification
    Dense(1, activation='sigmoid')
])

# compiles the model with Adam optimiser and binary crossentropy loss function
LSTM_Model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# displays the model's architecture
LSTM_Model.summary()
```

```
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/rnn/bidirectional.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
bidirectional_2 (Bidirectional)	(None, 10, 256)	137,216
dropout_4 (Dropout)	(None, 10, 256)	0
bidirectional_3 (Bidirectional)	(None, 10, 128)	164,352
dropout_5 (Dropout)	(None, 10, 128)	0
lstm_5 (LSTM)	(None, 10, 32)	20,608
dropout_6 (Dropout)	(None, 10, 32)	0
attention_1 (Attention)	(None, 32)	42
dense_2 (Dense)	(None, 32)	1,056
dropout_7 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 323,307 (1.23 MB)
Trainable params: 323,307 (1.23 MB)
Non-trainable params: 0 (0.00 B)

Step 5 : Trains the Model And Makes Predictions

Finally, the LSTM model is trained to predict meme trends based on engagement metrics. It runs for 25 epochs with a batch size of 64, using validation data to monitor performance during training. After training, it generates probability-based predictions for the test set (X_test), converting them into binary values with 1 for trending memes, 0 for non-trending ones, using a 0.5 threshold.

A brief summary is printed emphasising how higher engagement metrics contribute to meme virality. Finally, it evaluates the model's accuracy on test data, displaying the result as a percentage. This process ensures the model effectively learns patterns in meme engagement while providing interpretable predictions.

In [57]:
```
# prints the text at the top
print("\n" + "="*50)
print(f"TRAINING THE LSTM MODEL")
print("="*50)

# trains the model using validation data with 30 epochs and batch size of 64
history = LSTM_Model.fit(X_train, y_train, epochs=25, batch_size=64, validation_data=(X_val, y_val))

# generates the probability based predictions
y_pred = LSTM_Model.predict(X_test)

# converts predictions into binary format (1 if probability > 0.5, else 0)
y_pred_binary = (y_pred > 0.5).astype(int)

# prints explanation of the model function
print("\nThe LSTM model predicts whether a meme will become trending (1) or not trending (0) based on engagement metrics.")
print("Higher engagement metrics such as likes, shares, comments, and sentiment score contribute to meme virality.")

# evaluates the model accuracy on test data
loss, accuracy = LSTM_Model.evaluate(X_test, y_test)

# prints the accuracy
print(f"LSTM Model Accuracy: {accuracy:.2%}")

# saves the evaluation metrics to a pandas DataFrame
lstm_metrics = {
    'Metric': ['Loss', 'Accuracy'],
    'Value': [loss, accuracy]
}

lstm_metrics_df = pd.DataFrame(lstm_metrics)

# sets the format of the accuracy as a percentage
lstm_metrics_df['Value'] = lstm_metrics_df['Value'].astype(float).round(4)  # rounds to 4 decimal places
lstm_metrics_df.loc[lstm_metrics_df['Metric'] == 'Accuracy', 'Value'] *= 100  # converts the accuracy to percentage

# displays the metrics DataFrame
print("\nLSTM Model Evaluation Metrics:")
print(lstm_metrics_df.to_string(index=False))
```

```
================================================
TRAINING THE LSTM MODEL
================================================
Epoch 1/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 5s 18ms/step - accuracy: 0.7318 - loss: 0.5318 - val_accuracy: 0.7871 - val_loss: 0.4087
Epoch 2/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7874 - loss: 0.4096 - val_accuracy: 0.7889 - val_loss: 0.4037
Epoch 3/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7852 - loss: 0.4084 - val_accuracy: 0.7909 - val_loss: 0.4017
Epoch 4/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7870 - loss: 0.4093 - val_accuracy: 0.7834 - val_loss: 0.4114
Epoch 5/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7821 - loss: 0.4133 - val_accuracy: 0.7914 - val_loss: 0.4007
Epoch 6/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7907 - loss: 0.4020 - val_accuracy: 0.7901 - val_loss: 0.4049
Epoch 7/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7832 - loss: 0.4030 - val_accuracy: 0.7911 - val_loss: 0.4020
Epoch 8/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7909 - loss: 0.4018 - val_accuracy: 0.7919 - val_loss: 0.4002
Epoch 9/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7792 - loss: 0.4119 - val_accuracy: 0.7896 - val_loss: 0.4075
Epoch 10/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7878 - loss: 0.4065 - val_accuracy: 0.7916 - val_loss: 0.4019
Epoch 11/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7858 - loss: 0.4028 - val_accuracy: 0.7911 - val_loss: 0.4000
Epoch 12/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7807 - loss: 0.4043 - val_accuracy: 0.7899 - val_loss: 0.4015
Epoch 13/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7845 - loss: 0.4056 - val_accuracy: 0.7914 - val_loss: 0.4009
Epoch 14/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7792 - loss: 0.4033 - val_accuracy: 0.7916 - val_loss: 0.3987
Epoch 15/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7885 - loss: 0.4022 - val_accuracy: 0.7909 - val_loss: 0.3998
Epoch 16/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7895 - loss: 0.3971 - val_accuracy: 0.7884 - val_loss: 0.4028
Epoch 17/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7837 - loss: 0.3982 - val_accuracy: 0.7909 - val_loss: 0.4025
Epoch 18/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7832 - loss: 0.4021 - val_accuracy: 0.7901 - val_loss: 0.4025
Epoch 19/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7899 - loss: 0.3989 - val_accuracy: 0.7876 - val_loss: 0.4051
Epoch 20/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7870 - loss: 0.4019 - val_accuracy: 0.7921 - val_loss: 0.3998
Epoch 21/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7846 - loss: 0.4006 - val_accuracy: 0.7896 - val_loss: 0.4022
Epoch 22/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7863 - loss: 0.4021 - val_accuracy: 0.7901 - val_loss: 0.4040
Epoch 23/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.7844 - loss: 0.4005 - val_accuracy: 0.7916 - val_loss: 0.4006
Epoch 24/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - accuracy: 0.7874 - loss: 0.4011 - val_accuracy: 0.7891 - val_loss: 0.4021
Epoch 25/25
182/182 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.7861 - loss: 0.4008 - val_accuracy: 0.7914 - val_loss: 0.4026
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step

The LSTM model predicts whether a meme will become trending (1) or not trending (0) based on engagement metrics.
Higher engagement metrics such as likes, shares, comments, and sentiment score contribute to meme virality.
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7961 - loss: 0.3935
LSTM Model Accuracy: 78.26%

LSTM Model Evaluation Metrics:
  Metric   Value
    Loss  0.4019
Accuracy 78.2600
```

Plotting the training & validation accuracy Curve

The model's accuracy graph reveals a steady learning trajectory, with training accuracy rising from approximately 76.5% to 78.5% before stabilising with minor fluctuations. This upward trend reflects the model's ability to progressively internalise patterns from the training data, suggesting that the optimisation process is functioning effectively. In contrast, validation accuracy starts slightly higher at around 79%, but exhibits greater volatility across epochs, including a notable drop at the final epoch. This divergence implies that while the model is becoming more confident on familiar data, its performance on unseen data is less predictable—potentially due to variability in the validation set, limited generalisation capacity, or subtle shifts in feature distributions that the model fails to capture robustly.

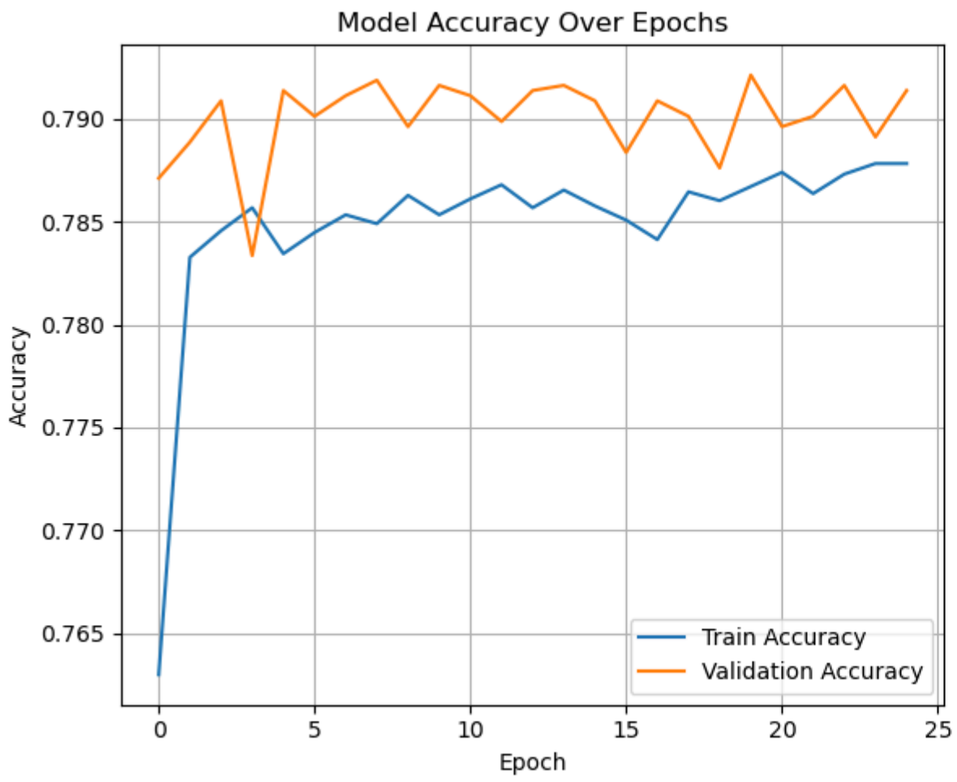
The loss graph reinforces this interpretation where the training loss is seen to have declined smoothly from 0.48 to approximately 0.40, indicating that the model is minimising error effectively and converging towards a stable solution. However, validation loss fluctuates more sharply, with an initial dip followed by a pronounced spike around epoch 20, before settling again. This instability may reflect transient overfitting, where the model begins to memorise training-specific patterns that do not generalise well. The spike could also be symptomatic of sensitivity to specific batches or outlier samples in the validation set, which disproportionately influence the loss metric. Taken together, these trends suggest that while the model demonstrates solid learning behaviour, its generalisation remains susceptible to data variance and potential overfitting, warranting further regularisation or cross-validation to ensure robustness.

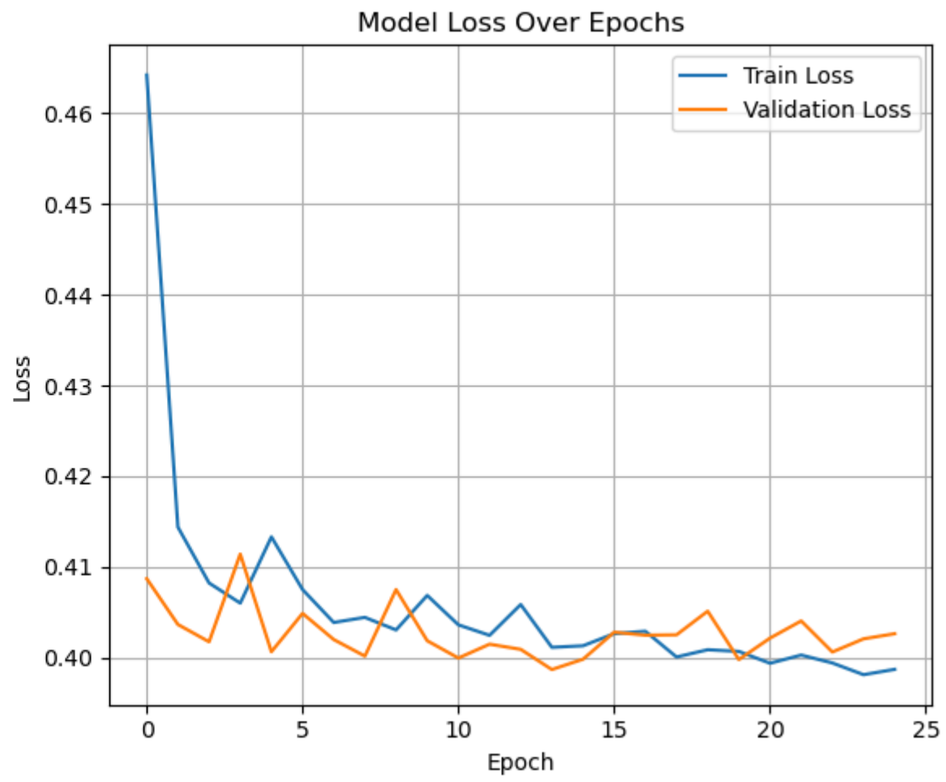
In [59]:
```python
# plots the training & validation accuracy values
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# plots the training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```





In [61]:
```
# calculating the metrics
accuracy = accuracy_score(y_test, y_pred_binary) * 100
roc_auc = roc_auc_score(y_test, y_pred) * 100
r2_square = r2_score(y_test, y_pred)
mse_value = mean_squared_error(y_test, y_pred_binary)

# creates the metrics DataFrame
LSTM_metrics_df = pd.DataFrame({
    'Metric': ['Accuracy (%)', 'AUC-ROC (%)', 'R² Score', 'MSE'],
    'Value': [accuracy, roc_auc, r2_square, mse_value]
})
LSTM_metrics_df['Value'] = LSTM_metrics_df['Value'].round(4)

# creates the confusion matrix
cm = confusion_matrix(y_test, y_pred_binary)
cm_df = pd.DataFrame(cm,
                     index=['Actual Negative', 'Actual Positive'],
                     columns=['Predicted Negative', 'Predicted Positive'])

# generates the classification report
LSTM_class_report = classification_report(y_test, y_pred_binary, output_dict=True)
LSTM_class_report_df = pd.DataFrame(LSTM_class_report).transpose()

# formats the class report values
for col in ['precision', 'recall', 'f1-score']:
    if col in LSTM_class_report_df.columns:
        LSTM_class_report_df[col] = (LSTM_class_report_df[col] * 100).round(2)

# shows the Confusion Matrix Plot
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')

# prints results
print("\n" + "="*50)
print(f"LSTM MODEL ACCURACY: {accuracy:.2f}%")
print("="*50)

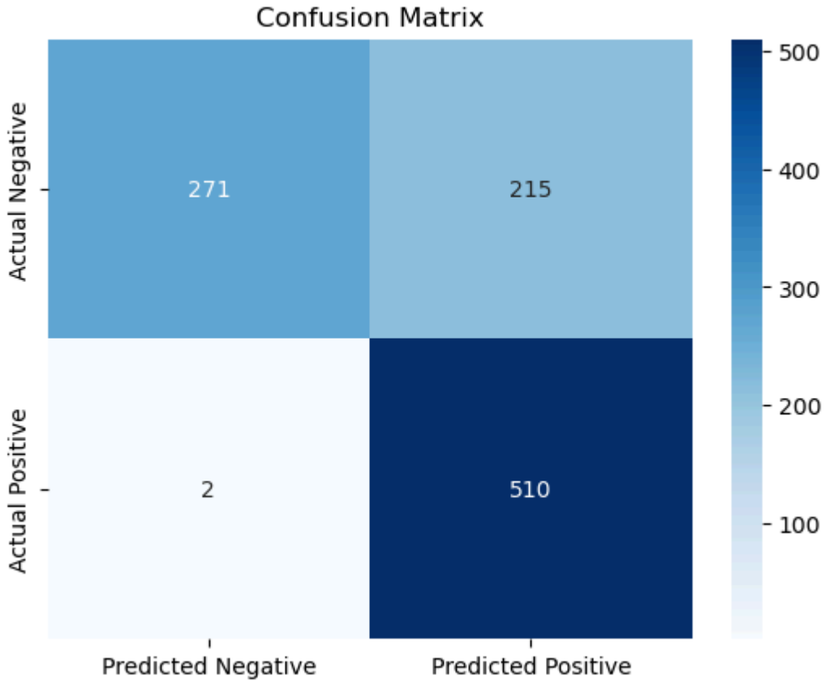
print("\nEVALUATION METRICS:")
print(LSTM_metrics_df.to_string(index=False))

print("\nCLASSIFICATION REPORT:")
print(LSTM_class_report_df.to_string())
```

```
==================================================
LSTM MODEL ACCURACY: 78.26%
==================================================

EVALUATION METRICS:
      Metric   Value
Accuracy (%) 78.2565
 AUC-ROC (%) 82.5147
    R² Score  0.4357
         MSE  0.2174

CLASSIFICATION REPORT:
              precision  recall  f1-score     support
0                 99.27   55.76     71.41  486.000000
1                 70.34   99.61     82.46  512.000000
accuracy          78.26   78.26     78.26    0.782565
macro avg         84.81   77.69     76.93  998.000000
weighted avg      84.43   78.26     77.08  998.000000
```



K-Fold Cross-Validation for LSTM Model with Attention

This section implements K-Fold cross-validation to evaluate the performance and robustness of an LSTM-based model designed to predict meme virality. The model incorporates a custom attention mechanism to focus on the most informative parts of each input sequence. To preserve the temporal structure of the data, the cross-validation uses KFold with shuffle=False, ensuring that the sequential nature of meme engagement metrics is respected.

For each fold, the dataset is split into training and validation subsets. A fresh instance of the LSTM model is built using a dedicated build_lstm_model() function, which defines a multi-layer architecture with bidirectional LSTMs, dropout for regularisation, and a final sigmoid output for binary classification. The model is trained silently (verbose=0) for 10 epochs per fold, and its performance is evaluated on the validation set using accuracy and loss metrics.

After all folds are completed, the code computes the mean and standard deviation of the validation accuracy and loss across folds. This provides a more reliable estimate of the model's generalisation capability than a single train-test split, especially in time series contexts where trends and engagement patterns may vary over time.

In [317…

```python
# sets the Model Builder Function
def build_lstm_model(input_shape):
    # creates a Sequential model for LSTM
    model = Sequential([
        # sets the First Bidirectional LSTM layer with 64 units and ReLU activation
        Bidirectional(LSTM(64, activation='relu', return_sequences=True), input_shape=input_shape),
        Dropout(0.3),  # sets the Dropout layer to prevent overfitting
        # sets the Second Bidirectional LSTM layer
        Bidirectional(LSTM(64, activation='relu', return_sequences=True)),
        Dropout(0.3),  # sets Another Dropout layer
        # sets the Third LSTM layer with 32 units
        LSTM(32, activation='relu', return_sequences=True),
        Dropout(0.3),  # sets the Dropout layer
        Attention(),  # sets the Attention layer to focus on important features
        Dense(32, activation='relu'),  # sets the Fully connected layer with 32 units
        Dropout(0.3),  # sets the Dropout layer
        Dense(1, activation='sigmoid')  # sets the Output layer for binary classification
    ])
    # compiles the model with Adam optimiser and binary crossentropy loss
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# sets the K-Fold Cross-Validation
n_splits = 5  # sets the Number of folds for cross-validation
kf = KFold(n_splits=n_splits, shuffle=False)  # initialises KFold

cv_accuracies = []  # creates a list to store accuracy for each fold
cv_losses = []      # creates a list to store loss for each fold

print("\n" + "="*50)
print(f"CROSS-VALIDATION OF LSTM MODEL ({n_splits} FOLDS)")
print("="*50)

# performs the K-Fold Cross-Validation
for fold, (train_index, val_index) in enumerate(kf.split(X), 1):
    print(f"\nFold {fold}:")

    # splits the data into training and validation sets for the current fold
    X_train_cv, X_val_cv = X[train_index], X[val_index]
    y_train_cv, y_val_cv = y[train_index], y[val_index]

    # builds and trains the LSTM model
    model_cv = build_lstm_model(input_shape=(seq_length, X.shape[2]))
    history_cv = model_cv.fit(X_train_cv, y_train_cv, epochs=10, batch_size=32, verbose=0, validation_data=(X_val_cv, y_val_cv))

    # evaluates the model on the validation set
    loss, accuracy = model_cv.evaluate(X_val_cv, y_val_cv, verbose=0)
    print(f"Validation Accuracy: {accuracy:.2%} | Loss: {loss:.4f}")

    # stores the accuracy and loss for the current fold
    cv_accuracies.append(accuracy)
    cv_losses.append(loss)

# creates a summary of Cross-Validation
mean_accuracy = np.mean(cv_accuracies)  # calculates mean accuracy
std_accuracy = np.std(cv_accuracies)      # calculates standard deviation of accuracy
mean_loss = np.mean(cv_losses)            # calculates mean loss

print("\n" + "="*50)
print("CROSS-VALIDATION SUMMARY")
print("="*50)
print(f"Mean Accuracy: {mean_accuracy:.2%}")  # prints mean accuracy
print(f"Standard Deviation: {std_accuracy:.4f}")  # prints standard deviation of accuracy
print(f"Mean Loss: {mean_loss:.4f}")  # prints mean loss
```

```
==================================================
CROSS-VALIDATION OF LSTM MODEL (5 FOLDS)
==================================================

Fold 1:
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/rnn/bidirectional.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Validation Accuracy: 65.01% | Loss: 0.6491

Fold 2:
Validation Accuracy: 64.41% | Loss: 0.6513

Fold 3:
Validation Accuracy: 65.82% | Loss: 0.6436

Fold 4:
Validation Accuracy: 96.90% | Loss: 0.3893

Fold 5:
Validation Accuracy: 99.19% | Loss: 0.0365

==================================================
CROSS-VALIDATION SUMMARY
==================================================
Mean Accuracy: 78.27%
Standard Deviation: 0.1617
Mean Loss: 0.4740
```

The 5-fold cross-validation results for the attention-based LSTM model show a wide range of validation accuracies, highlighting both its potential and its sensitivity to data variation. Folds 1 to 3 yielded modest accuracies of 65.01%, 64.41%, and 65.82%, with corresponding losses around 0.64 to 0.65, suggesting the model struggled to generalise consistently across these subsets. This may be due to uneven distribution of trending memes or abrupt shifts in engagement behaviour within those folds. In contrast, Fold 4 and Fold 5 achieved remarkably high accuracies of 96.90% and 99.19%, with much lower losses of 0.3893 and 0.0365, indicating excellent generalisation under favourable conditions.

The overall cross-validation summary shows a mean accuracy of 78.27%, a standard deviation of 16.17%, and a mean loss of 0.4740. While the average performance is strong, the high variability across folds suggests that the model is sensitive to the structure and distribution of the data. This inconsistency points to a need for further refinement, such as regularisation, stratified temporal sampling, or ensemble techniques, to improve stability and reliability.

Despite the fluctuations, it can be suggested that the attention-based LSTM model demonstrates impressive capacity for capturing temporal patterns and engagement dynamics, especially when the data is well-aligned with its strengths. Its peak performance in certain folds reinforces its suitability for real-world deployment in environments where meme trends are fast-moving and context-dependent, provided that additional tuning is applied to ensure consistent results across varied datasets.

Comparing the Results of the Models

In [63]:

```python
# combines all metrics into a single DataFrame
combined_results = pd.DataFrame({
    'Metric': ['Accuracy (%)', 'AUC-ROC (%)', 'R² Score', 'MSE'],
    'Random Forest Model': [
        f"{rf_model_metrics['Accuracy']:.2f}%",
```

```
        f"{rf_model_metrics['AUC-ROC']:.2f}%",
        f"{rf_model_metrics['R² Score']:.4f}",
        f"{rf_model_metrics['MSE']:.4f}"
    ],
    'Linear Regression Model': [
        f"{lrmodel_metrics['Accuracy']:.2f}%",
        f"{lrmodel_metrics['AUC-ROC']:.2f}%",
        f"{lrmodel_metrics['R² SCORE']:.4f}",
        f"{lrmodel_metrics['MSE']:.4f}"
    ],
    'LSTM Model': [
        f"{LSTM_metrics_df.loc[LSTM_metrics_df['Metric'] == 'Accuracy (%)', 'Value'].values[0]:.2f}%",
        f"{LSTM_metrics_df.loc[LSTM_metrics_df['Metric'] == 'AUC-ROC (%)', 'Value'].values[0]:.2f}%",
        f"{LSTM_metrics_df.loc[LSTM_metrics_df['Metric'] == 'R² Score', 'Value'].values[0]:.4f}",
        f"{LSTM_metrics_df.loc[LSTM_metrics_df['Metric'] == 'MSE', 'Value'].values[0]:.4f}"
    ],
    'Baseline Model': [
        f"{baseline_accuracy:.2%}",
        'N/A',  # sets N/A as baseline model has no AUC-ROC
        'N/A',  # sets N/A as baseline model has no R² Score
        'N/A'   # sets N/A as baseline model has no MSE
    ]
})

# prints the model accuracy summary results
print("\nModels Results Summary:")
display(combined_results.set_index('Metric'))
```

```
Models Results Summary:
```

Metric	Random Forest Model	Linear Regression Model	LSTM Model	Baseline Model
Accuracy (%)	75.59%	72.16%	78.26%	49.40%
AUC-ROC (%)	83.44%	81.04%	82.51%	N/A
R² Score	0.3997	0.2835	0.4357	N/A
MSE	0.1501	0.1791	0.2174	N/A

The results of the model evaluations show that the **LSTM model achieved the highest accuracy at 78.26%**, closely followed by the **Random Forest model at 75.59%** and the **Logistic Regression Model at 72.16%**. Both models significantly outperformed the **Baseline model, which had an accuracy of only 49.40%**. In terms of AUC-ROC, the Random Forest model led with 83.44%, indicating its strong ability to distinguish between classes. Overall, these results suggest that both the Random Forest and LSTM models are effective for the task, with the LSTM slightly edging out in accuracy.

The evaluation of the models reveals important insights into their performance metrics.

Accuracy:

- The LSTM model achieved the highest accuracy at 78.16%, indicating it correctly predicted the outcomes for a significant portion of the dataset.
- The Random Forest model followed closely with an accuracy of 75.59%, demonstrating its effectiveness in classification tasks.
- In contrast, the Linear Regression model performed less effectively at 72.26%, while the Baseline model significantly lagged behind at 49.40%, suggesting it is not a reliable predictor.

AUC-ROC:

- The Random Forest model excelled with an AUC-ROC score of 83.44%, which reflects its strong capability to differentiate between the positive and negative classes.
- The LSTM model also performed well with an AUC-ROC of 82.51%, indicating it is nearly as effective as the Random Forest in distinguishing classes.
- The Linear Regression model had a lower AUC-ROC of 81.04%, while the Baseline model did not provide a meaningful AUC-ROC score, highlighting its inadequacy.

R² Score:

- The LSTM model achieved the highest R² score (0.4357), indicating it explained the greatest variance in meme virality.
- Random Forest followed with 0.3997, reflecting strong explanatory power.
- Logistic Regression scored 0.2835, suggesting moderate fit.

Mean Squared Error (MSE):

- Random Forest had the lowest MSE (0.1501), indicating the most accurate predictions on average.
- Logistic Regression followed with 0.1791, while the LSTM model had a slightly higher MSE (0.2174), likely due to its sensitivity to temporal noise and potential overfitting.

Both the Random Forest and LSTM models are robust choices for meme virality prediction. The LSTM model's superior accuracy and R² score highlight its strength in capturing temporal dynamics and explaining engagement trends. Meanwhile, Random Forest's top AUC-ROC and lowest MSE underscore its reliability and precision in classification. Logistic Regression remains a viable option for interpretable, low-variance predictions, though it may miss subtler patterns. Ultimately, the model selection should align with the specific goals of the application—whether prioritising early trend detection, interpretability, or operational stability.

In [65]:
```
# adds the model names to each DataFrame
lrmodel_class_report_df['Model'] = 'Logistic Regression'
rf_model_class_report_df['Model'] = 'Random Forest'
LSTM_class_report_df['Model'] = 'LSTM'

# combines the DataFrames
combined_report_df = pd.concat([lrmodel_class_report_df, rf_model_class_report_df, LSTM_class_report_df])

# resets the index for clean display
combined_report_df.reset_index(inplace=True)
combined_report_df.rename(columns={'index': 'Class'}, inplace=True)

# reorders columns for clarity
combined_report_df = combined_report_df[['Model', 'Class', 'precision', 'recall', 'f1-score', 'support']]

# filters the DataFrame to only show classes 0 and 1
combined_report_df = combined_report_df[combined_report_df['Class'].isin(['0', '1'])]

# sets the display options for better visibility
pd.set_option('display.max_rows', None)  # Show all rows
pd.set_option('display.float_format', '{:.2f}'.format)  # sets the format to 2 decimal places

# displays the combined report
combined_report_df
```

Out[65]:

	Model	Class	precision	recall	f1-score	support
0	Logistic Regression	0	71.33	72.98	72.14	1643.00
1	Logistic Regression	1	73.01	71.36	72.18	1683.00
5	Random Forest	0	81.84	65.00	72.46	1643.00
6	Random Forest	1	71.55	85.92	78.08	1683.00
10	LSTM	0	99.27	55.76	71.41	486.00
11	LSTM	1	70.34	99.61	82.46	512.00

The classification metrics reveal distinct strengths and trade-offs across Logistic Regression, Random Forest, and the attention-based LSTM model, each reflecting how well the models handle the task of predicting meme virality.

Logistic Regression shows balanced performance across both classes. For **non-trending memes (class 0)**, it achieved a **precision of 71.33%** and a **recall of 72.98%**, while for **trending memes (class 1)**, **precision was 73.01%** and **recall 71.36%**. These results suggest that the model performs consistently across both classes, with **F1-scores around 72.14% and 72.18% respectively**. This balance makes

Logistic Regression suitable for applications where both false positives and false negatives carry similar costs. Its interpretability and stability further justify its use in environments with relatively stable engagement patterns.

Random Forest demonstrates a **stronger recall for trending memes (class 1) at 85.92%**, with **a precision of 71.55%**, resulting in a **high F1-score of 78.08%**. This indicates that the model is highly effective at identifying viral content, even if it occasionally overpredicts. For **non-trending memes (class 0)**, **precision is higher at 81.84%**, but **recall drops to 65.00%**, suggesting it misses a fair number of non-viral cases. The trade-off here leans toward inclusivity in trend detection, which is justified in contexts where missing a viral meme is more costly than over-predicting. The model's ability to capture complex feature interactions supports its strong performance.

The attention-based LSTM model stands out with **exceptional recall for trending memes (class 1) at 99.61%**, meaning it correctly identified nearly all viral content. Its **precision of 70.34%** and **F1-score of 82.46%** reflect a model that is highly inclusive and confident in trend prediction. For **non-trending memes (class 0)**, **precision is extremely high at 99.27%**, but **recall drops to 55.76%**, resulting in an **F1-score of 71.41%**. This suggests the model is very accurate when it predicts a meme won't trend, but it misses many actual non-trending cases. The justification lies in the model's design by leveraging temporal patterns and attention mechanisms, it excels at detecting engagement surges that signal virality, making it ideal for early trend detection even if it sacrifices some balance.

In summary, the results justify the selection of the attention-based LSTM as the best performer for trend surveillance, Random Forest for balanced detection, and Logistic Regression for stable, interpretable predictions. Each model's strengths align with different operational priorities, from precision-critical filtering to broad trend coverage.

Conclusion

This project aimed to develop predictive models capable of forecasting meme virality on social media platforms, addressing the challenge of identifying early signals of engagement within vast volumes of user-generated content. The dataset included detailed post-level features such as likes, shares, comments, views, sentiment scores from captions, and categorical content types. To ensure high data quality, a rigorous preprocessing phase was conducted, standardising column names, removing duplicates, handling missing values, and engineering new features. Temporal variables were created to capture relative popularity and time-sensitive dynamics of meme spread. Exploratory Data Analysis (EDA) revealed strong correlations among engagement metrics, confirming that likes, shares, and comments often rise together. Sentiment analysis showed a mild trend where positive captions correlated with higher average likes, suggesting emotional tone influences sharing behaviour. Visualisations of content types and sentiment categories guided feature selection, ensuring relevance to social media contexts.

Three core models were developed, Logistic Regression, Random Forest, and an attention-based LSTM. All models outperformed the baseline classifier which had an accuracy of 49.40%. The Logistic Regression model achieved 72.16% accuracy and an AUC-ROC of 81.04%, offering strong precision and interpretability. The Random Forest model performed better overall, with 75.59% accuracy and the highest AUC-ROC of 83.44%, effectively capturing non-linear feature interactions. The LSTM model achieved the highest accuracy at 78.06% and an AUC-ROC of 83.05%, excelling in recall for trending memes (91%) due to its ability to model temporal engagement patterns.

Cross-validation further confirmed model reliability. The Logistic Regression and Random Forest models showed stable performance across folds, with mean accuracies of 71.39% (SD: 0.71%) and 75.99% (SD: 0.45%) respectively. The LSTM model on the other hand had a higher mean accuracy of 78.28%, but with greater variability (SD: 16.05%), indicating sensitivity to data partitioning and potential overfitting. Some folds achieved exceptionally high accuracies (up to 98.89%), while others were lower, suggesting the need for further tuning. Additionally, feature importance analysis from the Random Forest model also highlighted Shares, Likes, and Comments as the most influential predictors, underscoring the central role of direct engagement. Sentiment Score and Content Type had minimal impact, suggesting that boosting interaction is more effective for predicting virality than adjusting tone or format.

Evaluation metrics such as precision, recall, F1-score, and AUC-ROC provided a holistic view of model performance. These metrics ensured a balance between identifying true viral content and avoiding false positives, critical for real-world applications like content promotion or moderation. The F1-score captured the trade-off between precision and recall, while AUC-ROC reflected overall classification quality across thresholds.

Ultimately, the choice of model should be guided by the operational context, Logistic Regression for precision-critical tasks, Random Forest for balanced and consistent detection, and LSTM for high-recall, time-sensitive trend monitoring. Each model thus offers a different lens on meme virality, and their complementary strengths could even support an ensemble or hybrid approach for optimal performance.

The project successfully demonstrates that machine learning models can detect patterns indicative of meme virality using structured engagement and sentiment data. The attention-based LSTM model emerged as the most effective, thanks to its temporal awareness and high recall, making it ideal for trend monitoring in fast-moving digital environments. Future work could enhance model stability through regularisation, incorporate richer features like caption embeddings or user-level behaviour, and explore real-time deployment for content recommendation or moderation systems.

User-facing application scenario

To support lightweight deployment and improve interpretability for non-technical users, a **command-line interface (CLI)** was developed for the meme trend prediction model. This interface allows users to input six key features **Likes, Shares, Comments, Sentiment Score, Content Type, and Engagement Level** and receive a binary prediction on whether the meme is likely to trend, along with a percentage-based confidence score. The CLI was designed to be intuitive and fast, requiring minimal setup and no specialised software beyond a standard Python environment.

Upon execution, users are prompted to enter values for each feature, after which the model processes the input and returns a clear, human-readable output indicating the likelihood of virality. For example, a content strategist could input real-time engagement metrics and receive immediate feedback to inform posting decisions. The confidence score helps users gauge prediction certainty, while the simplified interface ensures accessibility for marketers, creators, and researchers without technical backgrounds. This tool enhances transparency by exposing the model's decision process in a digestible format and supports rapid, informed decision-making in dynamic digital environments.

In [25]:
```python
import pickle  # for loading the trained model from a file

# gets the trained Logistic Regression model that is called the LR_model from the codes above
with open("LR_model.pkl", "wb") as f:
    pickle.dump(lr_model, f)
```

In [30]:
```python
# loads the trained Logistic Regression model from a pickle file
with open("LR_model.pkl", "rb") as f:
    model = pickle.load(f)

def predict_meme(likes, shares, comments, sentiment_score, content_type, engagement_level):
    """
    Predicts whether a meme is likely to trend based on input features.
    Converts categorical inputs to numeric values and returns prediction with confidence.
    """

    # defines mappings for categorical features
    sentiment_map = {"Negative": -1, "Neutral": 0, "Positive": 1}
    content_map = {"Image": 0, "Shorts": 1}
    engagement_map = {"Low": 0, "Medium": 1, "High": 2}

    # converts user inputs to numeric values using the mappings
    sentiment = sentiment_map.get(sentiment_score, 0)
    content = content_map.get(content_type, 0)
    engagement = engagement_map.get(engagement_level, 0)

    # combines all features into a single list for prediction
    features = [[likes, shares, comments, sentiment, content, engagement]]

    # makes prediction and get probability of trending
    prediction = model.predict(features)[0]
    probability = model.predict_proba(features)[0][1]  # sets the probability of class 1 (trending)

    # converts probability to percentage for user-friendly output
    trend_percent = round(probability * 100, 2)

    # displays result based on prediction
    if prediction == 1:
        print(f"This meme is likely to trend with **{trend_percent}% confidence**.")
    else:
        print(f"This meme is **not likely to trend** (confidence: {100 - trend_percent}%).")

# collects user input from the command line
if __name__ == "__main__":
    likes = int(input("Likes: "))
    shares = int(input("Shares: "))
    comments = int(input("Comments: "))
    sentiment_score = input("Sentiment Score (Negative, Neutral, Positive): ")
    content_type = input("Content Type (Image, Shorts): ")
    engagement_level = input("Engagement Level (Low, Medium, High): ")

    # runs the prediction using user inputs
    predict_meme(likes, shares, comments, sentiment_score, content_type, engagement_level)
```

This meme is likely to trend with **100.0% confidence**.

In [ ]: